

Big Data Engineering Practice Exam

Module 02 – Storage Part 1 (pp. 121–219)

Question 1: In the GFS (Google File System) architecture, what is the primary responsibility of the GFS Master?

- A. Storing all file data chunks as plain Linux files.
- B. Caching file data to improve read performance for clients.
- C. Maintaining all file system metadata, such as namespaces and file-to-chunk mappings.
- D. Issuing data requests directly to chunkservers.

Answer: C. Maintaining all file system metadata, such as namespaces and file-to-chunk mappings.

Explanation: The GFS Master is responsible for all system-wide activities and maintains all metadata, which includes "Namespaces, ACLs, mappings from files to chunks, and current locations of chunks". The GFS Client issues data requests, and Chunkservers store chunks as plain files.

Reference: Module 02 - Storage Part 1 - "Maintains all file system metadata" / "Namespaces, ACLs, mappings from files to chunks, and current locations of chunks"

Question 2: GFS is designed with specific assumptions about workloads. Which of the following is a key assumption that influenced its design?

- A. Most files are small, and random writes are very common.
- B. Most files are multi-GB, and files are typically modified by appending, not random writes.
- C. Data consistency must be strictly POSIX-compliant.
- D. Node failures are rare and should be treated as exceptions.

Answer: B. Most files are multi-GB, and files are typically modified by appending, not random writes.

Explanation: The GFS design is optimized for "Huge files (multi-GB)". It assumes "Most files are modified by appending to the end" and "Random writes (and overwrites) are practically non-existent." It also assumes node failures happen frequently.

Reference: Module 02 - Storage Part 1 - "Huge files (multi-GB)" / "Most files are modified by appending to the end" / "Random writes (and overwrites) are practically non-existent"

Question 3: What is a primary advantage of GFS using a large chunk size (e.g., 64MB or 128MB)?

- A. It eliminates internal fragmentation for small files.
- B. It increases the amount of interaction a client needs to have with the master.
- C. It reduces the size of the metadata that must be stored on the master node.
- D. It ensures all replicas of a chunk are stored on the same rack.

Answer: C. It reduces the size of the metadata that must be stored on the master node.

Explanation: A large chunk size has several advantages, including: "Reduces the size of the metadata stored in master" and "Reduces clients' need to interact with master". A *disadvantage* is that it can lead to "Wasted space due to internal fragmentation" for small files.

Reference: Module 02 - Storage Part 1 - "Advantages" / "Reduces the size of the metadata stored in master" / "Reduces clients' need to interact with master"

Question 4: In GFS, how is the data flow separated from the control flow?

- A. The master handles all data flow, and chunkservers handle all control flow.
- B. Clients interact with the master for metadata (control flow) and directly with chunkservers for file operations (data flow).
- C. All client requests go through the master, which then proxies them to the chunkservers.
- D. Data flow and control flow are bundled into a single request to the primary chunkserver.

Answer: B. Clients interact with the master for metadata (control flow) and directly with chunkservers for file operations (data flow).

Explanation: GFS explicitly states that "Data flow is decoupled from control flow". "Clients interact with the master for metadata operations (control flow)" and "Clients interact directly with chunkservers for all files operations (data flow)".

Reference: Module 02 - Storage Part 1 - "Data flow is decoupled from control flow" / "Clients interact with the master for metadata operations (control flow)" / "Clients interact directly with chunkservers for all files operations (data flow)"

Question 5: During a GFS write operation (mutation), what is the role of the "primary" chunk replica?

- A. It is the only replica that stores the data; secondary replicas only store metadata.
- B. It determines the serial order for concurrent data updates and forwards this order to all secondary replicas.
- C. It collects data from all clients and sends it to the master for ordering.
- D. It holds a lease permanently until the file is deleted.

Answer: B. It determines the serial order for concurrent data updates and forwards this order to all secondary replicas.

Explanation: For consistency, one replica is designated as the primary. The "Primary defines the update order" for mutations. It "determines serial order for data instances in its buffer" and then "sends the serial order to the secondaries and tells them to perform the write".

Reference: Module 02 - Storage Part 1 - "Primary defines the update order" / "The primary determines serial order for data instances in its buffer and writes the instances in that order to the chunk" / "The primary sends the serial order to the secondaries"

Question 6: What mechanism does the GFS master use to detect a stale replica?

- A. It periodically checksums all replicas to find inconsistencies.
- B. It uses a chunk version number, which is increased when a new lease is granted to a chunk.
- C. It relies on clients to report stale data when a read request fails.
- D. Stale replicas are not possible in GFS due to its strong consistency model.

Answer: B. It uses a chunk version number, which is increased when a new lease is granted to a chunk.

Explanation: A replica can become stale if "a chunkserver fails and misses mutations... while it is down". To distinguish up-to-date and stale replicas, GFS uses a "Chunk version number". This version is "Increased when master grants new lease on the chunk" and is "Not increased if replica is unavailable".

Reference: Module 02 - Storage Part 1 - "Chunk replicas may become stale: if a chunkserver fails and misses mutations to the chunk while it is down" / "Chunk version number: Increased when master grants new lease on the chunk"

Question 7: When a GFS client initiates a read operation, what is the correct sequence of steps?

- A. Client -> Chunkserver (gets data) -> Master (validates data).
- B. Client -> Master (gets replica locations) -> Client (picks location) -> Chunkserver (gets data).
- C. Client -> Master (gets primary location) -> Primary Chunkserver (gets data).

D. Client -> All Chunkservers (broadcasts request) -> Closest Chunkserver (responds with data).

Answer: B. Client -> Master (gets replica locations) -> Client (picks location) -> Chunkserver (gets data).

Explanation: The client first "translates request and sends it to the master". The master "responds with chunk handle and replica locations". The client then "picks a location and sends the request" to that chunkserver, which "sends requested data to the client".

Reference: Module 02 - Storage Part 1 - "GFS client translates request and sends it to the master" / "The master responds with chunk handle and replica locations" / "The client picks a location and sends the request" / "The chunkserver sends requested data to the client"

Question 8: How does GFS handle file deletion?

A. The file and all its chunks are immediately and irretrievably deleted.

B. The file is renamed to a hidden name, and its chunks are removed during a lazy garbage collection process after a configurable time.

C. The master deletes the file's metadata, but the chunkserver must be manually cleaned.

D. Deletion is a metadata-only operation; data chunks are never physically deleted, only marked as "unused".

Answer: B. The file is renamed to a hidden name, and its chunks are removed during a lazy garbage collection process after a configurable time.

Explanation: Deletion is a "lazy garbage collection" operation. The master logs the deletion and "renames file to special name" (a hidden name with a deletion timestamp). The master "regularly removes hidden files older than 3 days (configurable)".

Reference: Module 02 - Storage Part 1 - "Actual lazy garbage collection" / "File renamed to a hidden name with deletion timestamp" / "Master regularly removes hidden files older than 3 days (configurable)"

Question 9: In GFS, concurrent appends to the same file are permitted. What consistency guarantee does GFS provide in this scenario?

A. The replicas will be byte-for-byte identical, but the file region may contain mingled fragments from different clients.

B. The appends from different clients will be interleaved at the byte level.

C. Only one client's append will succeed; all others will fail.

D. The file will become corrupted and must be re-written.

Answer: A. The replicas will be byte-for-byte identical, but the file region may contain mingled fragments from different clients.

Explanation: The primary replica enforces one update order, ensuring "We will have identical replicas". However, the file region itself "may end up containing mingled fragments from different clients". This state is described as "consistent but undefined".

Reference: Module 02 - Storage Part 1 - "We will have identical replicas" / "file region may end up containing mingled fragments from different clients" / "writes are consistent but undefined state in GFS"

Question 10: What is the purpose of a "shadow" master in GFS?

- A. It serves as a hot-standby that takes over write operations immediately if the primary master fails.
- B. It provides read-only access to the file system when the primary master is down.
- C. It manages all garbage collection and re-replication tasks to reduce load on the primary master.
- D. It stores a copy of all file data, while the primary master stores only metadata.

Answer: B. It provides read-only access to the file system when the primary master is down.

Explanation: The master state is replicated for reliability. The "Shadow (not mirror) master provides only read-only access to file system when primary master is down".

Reference: Module 02 - Storage Part 1 - "Shadow (not mirror) master provides only read-only access to file system when primary master is down"

Question 11: How does the GFS master's replica placement strategy aim to maximize reliability and bandwidth?

- A. By placing all replicas on the same high-performance machine.
- B. By placing all replicas on the same rack to minimize network latency.
- C. By spreading replicas across different machines and different racks.
- D. By letting the client choose where all replicas should be placed.

Answer: C. By spreading replicas across different machines and different racks.

Explanation: The goal of replica placement is to "Maximize data reliability, availability and bandwidth utilization". This is achieved by spreading replicas "across machines and racks". An example policy is "3rd replica on a different rack".

Reference: Module 02 - Storage Part 1 - "Maximize data reliability, availability and bandwidth utilization" / "Replicas spread across machines and racks" / "3rd replica on a different rack"

Question 12: Which of the following is *not* an operation managed by the GFS master?

- A. Namespace management and locking.
- B. Transferring file data from chunkservers to the client during a read.
- C. Garbage collection of deleted files.
- D. Re-balancing replicas for better disk utilization.

Answer: B. Transferring file data from chunkservers to the client during a read.

Explanation: The master is "never the bottleneck" because "Clients never read and write file data through the master". Clients only ask the master "which chunkservers to talk to". All other options (namespace management, re-balancing, garbage collection) are listed as master operations.

Reference: Module 02 - Storage Part 1 - "Clients never read and write file data through the master" / "Master Operations" (lists Namespace management, Rebalancing, Garbage collection)

Question 13: What is a key difference between HDFS and GFS, according to the slides?

- A. GFS has a NameNode, while HDFS has a Master.
- B. GFS supports random file writes, while HDFS only supports appends.
- C. HDFS has a default chunk size of 64MB, while GFS's is 128MB.
- D. GFS supports a single writer model, while HDFS supports multiple writers.

Answer: B. GFS supports random file writes, while HDFS only supports appends.

Explanation: The summary table "GFS vs. HDFS" lists several differences. One is that GFS allows "Random file writes possible," while HDFS is listed as "Only append is possible." The table also shows GFS Master = HDFS Namenode and GFS Chunkserver = HDFS DataNode.

Reference: Module 02 - Storage Part 1 - "GFS vs. HDFS" table / "Random file writes possible" / "Only append is possible"

Question 14: How does the GFS master use locks to manage concurrent operations in the namespace?

- A. It acquires a single global write lock for all operations to ensure serial order.

B. It uses a read lock on parent directories and a write lock on the specific file or directory being mutated.

C. It does not use locks; all namespace operations are handled by the primary chunkserver.

D. It relies on Chubby to manage all file system locks.

Answer: B. It uses a read lock on parent directories and a write lock on the specific file or directory being mutated.

Explanation: To allow for concurrency, "Each master operation acquires a set of locks before it runs". The mechanism is to "Read lock on internal nodes, and read/write lock on the leaf". For example, creating `/home/user/f1` requires a "read lock on the directory name `/home/user/`" and a "write lock on the file name `f1`".

Reference: Module 02 - Storage Part 1 - "Each master operation acquires a set of locks before it runs" / "Read lock on internal nodes, and read/write lock on the leaf" / "Example: creating multiple files..."

Question 15: What is the purpose of the "append" operation in GFS?

A. It allows a client to overwrite data in the middle of a file.

B. It allows multiple clients to append data to the same file concurrently.

C. It is a special operation used only by the master to append to its operation log.

D. It creates a low-cost snapshot of a file.

Answer: B. It allows multiple clients to append data to the same file concurrently.

Explanation: GFS supports typical file operations but also adds two special ones: `snapshot` and `append`. The `append` operation is specifically designed to "allow multiple clients to append data to the same file concurrently".

Reference: Module 02 - Storage Part 1 - "append: allow multiple clients to append data to the same file concurrently"

Question 16: A client pushes write data to all three replicas of a chunk. When does this data actually get written to the file?

A. Immediately upon arrival at each replica's buffer.

B. After the client sends a write command to the primary, which then determines the serial order and instructs all replicas to write.

C. After the client sends write commands to all three replicas simultaneously.

D. After the master receives the data, orders it, and forwards it to the primary.

Answer: B. After the client sends a write command to the primary, which then determines the serial order and instructs all replicas to write.

Explanation: The write process is a multi-step. First, the "client pushes write data to all locations", where it is "stored in chunkserver's internal buffers". Second, the "client sends write command to the primary". Third, the "primary determines serial order" and "sends the serial order to the secondaries and tells them to perform the write".

Reference: Module 02 - Storage Part 1 - "4. The client pushes write data to all locations" / "5. The client sends write command to the primary" / "6. The primary determines serial order" / "7. The primary sends the serial order to the secondaries"

Question 17: What is the primary concern raised in the slides regarding the design philosophy of large-scale DFS like GFS and HDFS?

- A. They are not fast enough for modern workloads.
- B. Their "business-first" design lacks participatory governance and ecological accountability.
- C. They are too difficult for small companies to implement.
- D. They do not use triple replication, which poses a data loss risk.

Answer: B. Their "business-first" design lacks participatory governance and ecological accountability.

Explanation: The slides critique the design philosophy as "Business-first: speed, cost, scale." This approach is noted as "Missing: participatory governance, equitable access, ecological accountability". The environmental impact of triple replication is also mentioned.

Reference: Module 02 - Storage Part 1 - "Design philosophy" / "Business-first: speed, cost, scale" / "Missing: participatory governance, equitable access, ecological accountability"

Question 18: How is data integrity (i.e., protection against corruption) handled in GFS?

- A. It is the client's responsibility to verify the data after reading.
- B. GFS checksums each 64KB block within a chunk, and this checksum is verified on read.
- C. The master periodically scans all chunks and compares them to find corrupted data.
- D. Data is written to all replicas, and a majority vote is used on read to find the correct data.

Answer: B. GFS checksums each 64KB block within a chunk, and this checksum is verified on read.

Explanation: For data integrity, GFS uses a "Checksum for each chunk divided into 64KB blocks". This "Checksum is checked every time an application reads the data" to detect corruption.

Reference: Module 02 - Storage Part 1 - "Data integrity" / "Checksum for each chunk divided into 64KB blocks" / "Checksum is checked every time an application reads the data"

Question 19: Which component in HDFS is analogous to the "Chunkserver" in GFS?

- A. NameNode
- B. DataNode
- C. Journal
- D. Client

Answer: B. DataNode

Explanation: The "GFS vs. HDFS" comparison table directly maps the components. The GFS "Chunkserver" (which stores data chunks) corresponds to the HDFS "DataNode".

Reference: Module 02 - Storage Part 1 - "GFS vs. HDFS" table (Chunkserver -> DataNode)

Question 20: When designing a community-focused DFS, the slides suggest an alternative replication strategy to GFS's standard triple replication. What is a key characteristic of this alternative?

- A. Adaptive replication that balances resilience with energy use, or using erasure coding.
- B. Replicating data at least 5 times to ensure zero data loss.
- C. Storing all replicas in a single, secure, community-controlled machine.
- D. Encrypting all replicas with different keys.

Answer: A. Adaptive replication that balances resilience with energy use, or using erasure coding.

Explanation: The slides propose an alternative replication strategy for a community-focused system. Instead of the high-energy-cost triple replication, it suggests "Adaptive replication: balance resilience with energy use" or "Use erasure coding or low-energy redundancy methods".

Reference: Module 02 - Storage Part 1 - "Replication Strategy" / "Adaptive replication: balance resilience with energy use" / "Use erasure coding or low-energy redundancy methods"

Module 03 – Storage Part 2 (pp. 220–344)

Question 1: In the context of the CAP theorem, what does "Partition Tolerance" refer to?

- A. The ability to always provide the most recent write to any read request.
- B. The ability to continue operation in the presence of network partitions.
- C. The ability for clients to always read and write data.
- D. The ability to split (partition) large tables into smaller, more manageable tablets.

Answer: B. The ability to continue operation in the presence of network partitions.

Explanation: Partition Tolerance is one of the three guarantees in the CAP theorem. It means the distributed system can "Continue the operation in the presence of network partitions".

Reference: Module 03 - Storage Part 2 - "Partition Tolerance" / "Continue the operation in the presence of network partitions"

Question 2: Which of the following best describes the "Eventual Consistency" model?

- A. All replicas are updated simultaneously, ensuring immediate consistency.
- B. The system guarantees that subsequent accesses will *always* return the updated value immediately.
- C. If no new updates are made, eventually all accesses will return the last updated value.
- D. The database is only consistent during planned maintenance events.

Answer: C. If no new updates are made, eventually all accesses will return the last updated value.

Explanation: Eventual consistency is a weaker model than strong consistency. It accepts a temporary "inconsistency window" where a read might not see the latest write. However, it guarantees that "If no new updates are made to the object, eventually all accesses will return the last updated value".

Reference: Module 03 - Storage Part 2 - "Eventual consistency" / "If no new updates are made to the object, eventually all accesses will return the last updated value"

Question 3: BigTable is described as a distributed multi-level map. What are its guarantees according to the CAP theorem?

- A. Availability and Partition Tolerance (AP)
- B. Strong Consistency and Availability (CA)
- C. Strong Consistency and Partition Tolerance (CP)
- D. It achieves all three: Consistency, Availability, and Partition Tolerance.

Answer: C. Strong Consistency and Partition Tolerance (CP)

Explanation: The slides explicitly state that BigTable's CAP tradeoff is "strong consistency and partition tolerance". This means it prioritizes ensuring all reads see the most recent data and can handle network partitions, but may sacrifice availability to do so.

Reference: Module 03 - Storage Part 2 - "BigTable" / "CAP: strong consistency and partition tolerance"

Question 4: In BigTable's architecture, what is the role of a "Tablet Server"?

- A. It assigns tablets to servers and handles schema changes.
- B. It stores the bootstrap location of BigTable data and discovers tablet servers.
- C. It manages a set of tablets, handles read/write requests to those tablets, and splits tablets that grow too large.
- D. It is a client-side library that caches tablet locations.

Answer: C. It manages a set of tablets, handles read/write requests to those tablets, and splits tablets that grow too large.

Explanation: The Tablet Server is a worker node in BigTable. Its key responsibilities include: "Each manages a set of tablets", "Handles read/write requests to tablets", and "Splits tablets when too large". The Master handles tablet assignment and schema changes.

Reference: Module 03 - Storage Part 2 - "Tablet server" / "Each manages a set of tablets (typically 10-1000 tablets/server)" / "Handles read/write requests to tablets" / "Splits tablets when too large"

Question 5: How do clients find the correct tablet server for a read/write operation in BigTable?

- A. They broadcast the request to all tablet servers, and the correct one responds.
- B. They contact the Master, which forwards the data request to the correct tablet server.
- C. They use a three-level hierarchy, starting with a file in Chubby that points to the METADATA table, which in turn locates the tablet.
- D. All clients maintain a complete, static map of all tablet locations.

Answer: C. They use a three-level hierarchy, starting with a file in Chubby that points to the METADATA table, which in turn locates the tablet.

Explanation: Tablet location is a "Three-level hierarchy". 1) A file "stored in Chubby that contains the location of the root tablet". 2) The "Root tablet contains location of all tablets in a special METADATA table". 3) The "METADATA table contains location of each tablet".

Reference: Module 03 - Storage Part 2 - "Three-level hierarchy" / "The first level is a file stored in Chubby that contains the location of the root tablet" / "Root tablet contains location of all tablets in a special METADATA table"

Question 6: What is an SSTable in the context of BigTable?

- A. A mutable, in-memory table for buffering recent writes.
- B. An immutable, sorted file of key-value pairs used internally to store BigTable data.
- C. A log file used by the Master to store schema changes.
- D. A service that ensures only one active master exists.

Answer: B. An immutable, sorted file of key-value pairs used internally to store BigTable data.

Explanation: SSTable (Sorted String Table) is the "SSTable file format used internally to store BigTable data". It is an "Immutable, sorted file of key-value pairs". Recent updates are first stored in an in-memory *memtable*.

Reference: Module 03 - Storage Part 2 - "SSTable" / "SSTable file format used internally to store BigTable data" / "Immutable, sorted file of key-value pairs"

Question 7: Which of the following is *not* a function of Chubby in the BigTable architecture?

- A. To ensure there is only one active master.
- B. To store the bootstrap location (location of the root tablet).
- C. To store BigTable schema information and access control lists.
- D. To store recently committed data updates (the memtable).

Answer: D. To store recently committed data updates (the memtable).

Explanation: Chubby is a lock service used for coordination. It is used to "Ensure there is only one active master", "Store bootstrap location of BigTable data", "Discover tablet servers", and "Store BigTable schema information and access control lists". The *memtable* is stored in the memory of the *tablet servers*.

Reference: Module 03 - Storage Part 2 - "Chubby" / "Ensure there is only one active master" / "Store bootstrap location of BigTable data" / "Store BigTable schema information and access control lists"

Question 8: How does BigTable achieve strong consistency for its data?

- A. By using eventual consistency with a very small inconsistency window.

B. By replicating data at the application layer and using client-side validation.

C. Data replication is handled by the underlying GFS, and only one tablet server is responsible for a given tablet at any time.

D. By requiring all reads and writes to go through the single master node.

Answer: C. Data replication is handled by the underlying GFS, and only one tablet server is responsible for a given tablet at any time.

Explanation: BigTable's strong consistency model relies on two facts: 1) "Only one tablet server is responsible for a given piece of data" (tablet), and 2) "Replication is handled on the GFS layer". This avoids the complexities of managing replica consistency at the BigTable layer.

Reference: Module 03 - Storage Part 2 - "Strong consistency" / "Only one tablet server is responsible for a given piece of data" / "Replication is handled on the GFS layer"

Question 9: A tablet server fails. How does the BigTable master detect this and reassign the tablets?

A. The failed tablet server sends a "goodbye" message to the master before shutting down.

B. The master periodically checks an exclusive lock in Chubby that each tablet server must hold; if the lock is lost, the server is considered down.

C. Clients report the failure to the master after a read/write request times out.

D. The master pings each tablet server, and if a ping fails, it immediately reassigns the tablets.

Answer: B. The master periodically checks an exclusive lock in Chubby that each tablet server must hold; if the lock is lost, the server is considered down.

Explanation: The master uses Chubby to track live tablet servers. "When a tablet server starts, it creates and acquires an exclusive lock in Chubby". The master "detects the status of the lock of each tablet server by checking periodically". If a server loses its lock, the master "is responsible for finding when tablet server is no longer serving its tablets and reassigning those tablets".

Reference: Module 03 - Storage Part 2 - "When a tablet server starts, it creates and acquires an exclusive lock in Chubby" / "Master detects the status of the lock of each tablet server by checking periodically" / "Master is responsible for finding when tablet server is no longer serving its tablets"

Question 10: What is the primary CAP theorem trade-off for Cassandra?

A. Consistency and Partition Tolerance (CP)

B. Availability and Partition Tolerance (AP)

C. Consistency and Availability (CA)

D. Cassandra is an ACID-compliant database and is not subject to the CAP theorem.

Answer: B. Availability and Partition Tolerance (AP)

Explanation: Unlike BigTable's CP model, Cassandra is explicitly designed to prioritize "availability and partition tolerance". This means it will continue to accept reads and writes even during network partitions, at the cost of strong consistency (favoring eventual consistency).

Reference: Module 03 - Storage Part 2 - "Cassandra" / "CAP: availability and partition tolerance"

Question 11: What partitioning strategy does Cassandra use to distribute data across the cluster?

A. A master node assigns key ranges to each worker node.

B. Consistent hashing.

C. All data is replicated to all nodes in the cluster.

D. Data is partitioned based on column families.

Answer: B. Consistent hashing.

Explanation: When data size exceeds a single machine, it must be partitioned. Cassandra uses "Consistent hashing for partitioning". This involves hashing "both data and node ids using the same hash function in a same id space".

Reference: Module 03 - Storage Part 2 - "Consistent hashing for partitioning" / "Hash both data and node ids using the same hash function in a same id space"

Question 12: Which of the following is NOT one of the four main NoSQL data models listed in the slides?

A. Key/Value

B. Document-oriented

C. Relational Model

D. Graph-based

Answer: C. Relational Model

Explanation: The slides list four main NoSQL data models: Key/Value (e.g., Dynamo), Column-oriented (e.g., BigTable, Cassandra), Document-oriented (e.g., MongoDB), and Graph-based (e.g., Neo4J). The Relational Model is what SQL databases use and is what NoSQL databases are an alternative to.

Reference: Module 03 - Storage Part 2 - "NoSQL Data Models" (slide title) / "Key/value pairs" / "Column-oriented store" / "Document-oriented store" / "Graph-based"

Question 13: A client writes data to BigTable. This update is first committed to a commit log. Where are the *recently* committed updates stored before being written to SSTables?

- A. In Chubby
- B. In the master node's memory
- C. In an in-memory "memtable" on the tablet server
- D. In a temporary SSTable

Answer: C. In an in-memory "memtable" on the tablet server

Explanation: When a write occurs, it is first "committed to a commit log". The "Recently committed updates are stored in memory - memtable" on the tablet server. "Older updates are stored in a sequence of SSTables".

Reference: Module 03 - Storage Part 2 - "Updates committed to a commit log" / "Recently committed updates are stored in memory - memtable" / "Older updates are stored in a sequence of SSTables"

Question 14: In the context of the social critique of large-scale systems, what is "context-aware sharding"?

- A. Sharding data based on its size to improve load balancing.
- B. A data sharding strategy designed to respect community boundaries.
- C. A technique to ensure all shards are stored in the same data center.
- D. Sharding data based on the time it was created.

Answer: B. A data sharding strategy designed to respect community boundaries.

Explanation: As part of a critique on how to reimagine systems for equity, "context-aware sharding" is proposed as an architectural change. It is defined as "data sharding designed to respect community".

Reference: Module 03 - Storage Part 2 - "Architecture" / "Context-aware sharding, i.e., data sharding designed to respect community"

Question 15: Which of the following is presented as a trade-off when choosing a decentralized, community-controlled NoSQL system (like CouchDB) over a hyperscale system (like BigTable)?

- A. Community systems have better performance but are less autonomous.

- B. Community systems offer global reach, while hyperscale systems prioritize local sovereignty.
- C. Community systems may have lower throughput/latency but provide more local control.
- D. Community systems have 24/7 professional support, while hyperscale systems rely on volunteers.

Answer: C. Community systems may have lower throughput/latency but provide more local control.

Explanation: The slides list several trade-offs. One is "Performance vs. Autonomy," which states that community-controlled systems may have "lower throughput/latency than hyperscale systems, but more local control". The other options are reversed (e.g., hyperscale systems offer global reach, and big tech offers 24/7 support).

Reference: Module 03 - Storage Part 2 - "Performance vs. Autonomy: lower throughput/latency than hyperscale systems, but more local control" / "Global Reach vs. Sovereignty: corporate clouds offer worldwide availability" / "Support vs. Self-reliance: bigTech = professional 24/7 support"

Question 16: What is the key difference between a Document-oriented store and a Column-oriented store?

- A. Column-oriented stores are for graphs, while Document-oriented stores are for key-value pairs.
- B. Document-oriented stores have a flexible schema where values can be complex documents (like JSON), while column-oriented stores group data by column.
- C. Column-oriented stores are strongly consistent (CP), while Document-oriented stores are available (AP).
- D. Document-oriented stores are immutable, while Column-oriented stores are mutable.

Answer: B. Document-oriented stores have a flexible schema where values can be complex documents (like JSON), while column-oriented stores group data by column.

Explanation: A column-oriented store "Store[s] and process[es] data by column". A document-oriented store is "Similar to a column-oriented store, but values can have complex documents" and is known for its "Flexible schema (XML, YAML, JSON, and BSON)".

Reference: Module 03 - Storage Part 2 - "Column-oriented store" / "Store and process data by column" / "Document-oriented store" / "Similar to a column-oriented store, but values can have complex documents" / "Flexible schema (XML, YAML, JSON, and BSON)"

Question 17: In BigTable's data model, what is the "basic unit of data access"?

- A. The Row
- B. The Tablet

C. The Column Family

D. The Cell

Answer: C. The Column

Explanation: While "Every read or write in a row is atomic", the slides state that the "basic unit of data access" is the Column. Columns are grouped into column families.

Reference: Module 03 - Storage Part 2 - "Column" / "The basic unit of data access"

Question 18: In BigTable, what is a "tablet"?

A. A contiguous range of rows stored together, which is split when it becomes too large.

B. A single SSTable file stored in GFS.

C. The in-memory buffer used to store recent writes (memtable).

D. A client-side library for caching data.

Answer: A. A contiguous range of rows stored together, which is split when it becomes too large.

Explanation: A tablet is defined as "contiguous ranges of rows stored together". The system automatically "split[s] tablets... when they become too large". Each tablet is "served by exactly one tablet server".

Reference: Module 03 - Storage Part 2 - "Tablet: contiguous ranges of rows stored together" / "Tablets are split by the system when they become too large"

Question 19: What is a key motivation for the development of NoSQL databases?

A. The need to enforce strict ACID transactions on a web-scale.

B. The high cost of SQL query languages.

C. The inability of RDBMS to scale for internet-scale data, high read-write rates, and frequent schema changes.

D. The desire for a single database standard that could handle all data models.

Answer: C. The inability of RDBMS to scale for internet-scale data, high read-write rates, and frequent schema changes.

Explanation: Traditional RDBMS "were not designed to be distributed". Web-based applications introduced new challenges (spikes) like "Internet-scale data size," "High read-write rates," and "Frequent schema changes" that RDBMS struggled with, leading to the development of NoSQL.

Reference: Module 03 - Storage Part 2 - "Web-based applications caused spikes" / "Internet-scale data size" / "High read-write rates" / "Frequent schema changes" / "RDBMS were not designed to be distributed"

Question 20: Which of the following systems is described as a "decentralized and offline-first" database that supports "local replicas" and "data sovereignty"?

- A. CouchDB
- B. GunDB
- C. OrbitDB
- D. Cassandra

Answer: A. CouchDB

Explanation: The slides present several "alternative" databases. CouchDB is explicitly described as "Decentralized and offline-first" and its "Local replicas support underserved regions and data sovereignty".

Reference: Module 03 - Storage Part 2 - "CouchDB" / "Decentralized and offline-first" / "Local replicas support underserved regions and data sovereignty"

Module 04 – A Crash Course on Scala (pp. 345–460)

Question 1: In Scala, what is the fundamental difference between a `val` and a `var`?

- A. `val` is for lazy evaluation; `var` is for eager evaluation.
- B. `val` is for mutable variables; `var` is for immutable values.
- C. `val` is for immutable values; `var` is for mutable variables.
- D. `val` can only be used inside a class; `var` can only be used inside a function.

Answer: C. `val` is for immutable values; `var` is for mutable variables.

Explanation: A `val` defines an "immutable" value (it cannot be reassigned), while a `var` defines a "mutable" variable (it can be reassigned). The slides recommend to "Always use immutable values by default".

Reference: Module 04 - A Crash Course on Scala - "Values: immutable" / "Variables: mutable" / "Always use immutable values by default, unless you know for certain they need to be mutable."

Question 2: Which of the following statements about Scala is correct according to the slides?

- A. Scala is a blend of object-oriented and functional programming that runs on the Java Virtual Machine.
- B. Scala is a purely functional language and cannot be used for object-oriented programming.
- C. Scala compiles to machine code and does not require a virtual machine.
- D. Scala allows inheritance from multiple classes, just like C++.

Answer: A. A blend of object-oriented and functional programming that runs on the Java Virtual Machine.

Explanation: Scala stands for "scalable language" and is explicitly described as a "blend of object-oriented and functional programming" that "Runs on the Java Virtual Machine." The slides also state it is a "pure object-oriented language" and "Scala allows the inheritance from just one class only."

Reference: Module 04 - A Crash Course on Scala - "A blend of object-oriented and functional programming." / "Runs on the Java Virtual Machine." / "Scala is a pure object-oriented language." / "Scala allows the inheritance from just one class only."

Question 3: What is a key difference between a Scala `Array` and a Scala `List` ?

- A. An `Array` is immutable, while a `List` is mutable.
- B. An `Array` is a fixed-size, mutable collection, while a `List` is an immutable, linked list.
- C. An `Array` can hold different types, while a `List` must hold elements of the same type.
- D. An `Array` has no duplicates, while a `List` allows duplicates.

Answer: B. An `Array` is a fixed-size, mutable collection, while a `List` is an immutable, linked list.

Explanation: The slides state that an `Array` is "A fixed-size sequential collection of elements of the same type" and is "Mutable". A `List` is also sequential but is "Immutable" and "Lists represent a linked list".

Reference: Module 04 - A Crash Course on Scala - "Arrays" / "A fixed-size sequential collection of elements of the same type" / "Mutable" / "Lists" / "A sequential collection of elements of the same type" / "Immutable" / "Lists represent a linked list"

Question 4: What is the result of the following Scala code?

```
val l1 = List(1, 2, 3)
val l2 = 0 :: l1
```

- A. `l2` will be `List(0, 1, 2, 3)` .
- B. `l2` will be `List(1, 2, 3, 0)` .
- C. The code will fail to compile because `l1` is immutable.
- D. `l2` will be `List(0)` and `l1` will be `List(1, 2, 3)` .

Answer: A. `l2` will be `List(0, 1, 2, 3)` .

Explanation: The `::` (cons) operator is shown as the method for "Adding an element to the head of a list". Because `List` is immutable, this operation returns a *new* list (`l2`) containing the new element (0) followed by all elements of the old list (`l1`).

Reference: Module 04 - A Crash Course on Scala - "// Adding an element to the head of a list" / "val l2 = 0 :: l1"

Question 5: What is a "higher-order function" in Scala?

- A. A function that is part of a `case class` .
- B. A function that can take other functions as arguments or return them as results.
- C. A function that can only operate on `Double` or `Float` types.
- D. A function defined with the `object` keyword, making it a singleton.

Answer: B. A function that can take other functions as arguments or return them as results.

Explanation: In Scala, "Functions are also objects". This means "it is possible to pass functions as arguments, to store them in variables, and to return them from other functions." A function that does this (like the `sum` function in the slides that is reimplemented) is a higher-order function.

Reference: Module 04 - A Crash Course on Scala - "Functions are also objects, so it is possible to pass functions as arguments, to store them in variables, and to return them from other functions." / "Reimplement the previous methods using higher-order functions."

Question 6: What is the key difference between the `map` and `foreach` collection operations in Scala?

- A. `map` returns a new list with the same number of elements, while `foreach` "returns nothing" (`Unit`).
- B. `map` only works on Lists, while `foreach` works on all collections.
- C. `map` modifies the collection in-place, while `foreach` returns a new collection.
- D. `map` is for filtering, while `foreach` is for transforming.

Answer: A. `map` returns a new list with the same number of elements, while `foreach` "returns nothing" (`Unit`).

Explanation: `map` "Evaluates a function over each element in the list, returning a list with the same number of elements." In contrast, `foreach` "is like map, but returns nothing." It is used for side effects, like printing.

Reference: Module 04 - A Crash Course on Scala - "map" / "Evaluates a function over each element in the list, returning a list with the same number of elements." / "foreach" / "It is like map, but returns nothing."

Question 7: Given `val myNumbers = List(1, 2, 3, 4)`, what is the result of `myNumbers.map(x => x * x)` ?

- A. `List(1, 4, 9, 16)`
- B. `List(1, 2, 3, 4)`
- C. `30`
- D. `Unit`

Answer: A. `List(1, 4, 9, 16)`

Explanation: The `map` operation evaluates the given function (an anonymous function `x => x * x`) over each element in the list and returns a new list with the results. This is based on the exercises in the slides asking for a `pow` function and writing it "inline in a map call".

Reference: Module 04 - A Crash Course on Scala - "map" / "Evaluates a function over each element in the list..." / "Declare a function, pow, that computes the second power of an int." / "Write the pow function inline in a map call, using closure notation."

Question 8: Given `val myNumbers = List(1, 2, 3, 4, 5)`, what is the result of `myNumbers.filter(x => x < 3)` ?

- A. `List(3, 4, 5)`

- B. `List(1, 2)`
- C. `List(true, true, false, false, false)`
- D. The code will not compile.

Answer: B. `List(1, 2)`

Explanation: The `filter` operation "Removes any elements where the function you pass in evaluates to false." In this case, it keeps only the elements that satisfy the predicate `x < 3`.

Reference: Module 04 - A Crash Course on Scala - "filter" / "Removes any elements where the function you pass in evaluates to false."

Question 9: What does the `flatMap` operation do?

- A. It is a combination of `map` and `flatten`; it maps a function and then concatenates the results.
- B. It filters a map to create a new, "flat" map.
- C. It converts a `List` to a `Map`.
- D. It finds the first element that matches a predicate function.

Answer: A. It is a combination of `map` and `flatten`; it maps a function and then concatenates the results.

Explanation: `flatMap` is a powerful operation that combines `map` and `flatten`. `flatten` "collapses one level of nested structure". `flatMap` "takes a function that works on the nested lists and then concatenates the results back together."

Reference: Module 04 - A Crash Course on Scala - "flatten" / "It collapses one level of nested structure." / "flatMap" / "It takes a function that works on the nested lists and then concatenates the results back together."

Question 10: What is a key characteristic of a Scala `Set`?

- A. It is a fixed-size, mutable collection.
- B. It is a collection of key/value pairs.
- C. It allows no duplicate elements.
- D. It is an immutable, linked list.

Answer: C. It allows no duplicate elements.

Explanation: A `Set` is a "sequential collection of elements of the same type" that has "No duplicates." It can be mutable or immutable.

Reference: Module 04 - A Crash Course on Scala - "Sets" / "A sequential collection of elements of the same type" / "Immutable and mutable" / "No duplicates."

Question 11: How is a Scala `Map` collection defined?

- A. A fixed-size sequential collection of elements of the same type.
- B. A collection of key/value pairs.
- C. A sequential, immutable linked list.
- D. A collection that does not allow duplicate elements.

Answer: B. A collection of key/value pairs.

Explanation: A `Map` is defined as "A collection of key/value pairs". An example is `val m2 = Map(1 -> "Carbon", 2 -> "Hydrogen")`.

Reference: Module 04 - A Crash Course on Scala - "Maps" / "A collection of key/value pairs"

Question 12: What is the difference between "Call-by-Value" and "Call-by-Name" for a function parameter?

- A. Call-by-Value passes a copy, while Call-by-Name passes a reference.
- B. Call-by-Value is for `val`, while Call-by-Name is for `var`.
- C. Call-by-Value determines the parameter's value *before* passing it; Call-by-Name determines it *when it is called* within the function.
- D. Call-by-Value is fast; Call-by-Name is slow.

Answer: C. Call-by-Value determines the parameter's value *before* passing it; Call-by-Name determines it *when it is called* within the function.

Explanation: Call-by-Value is the default, where "the value of the parameter is determined before it is passed to the function". In Call-by-Name, "the value of the parameter is not determined until it is called within the function."

Reference: Module 04 - A Crash Course on Scala - "Call-by-Value: the value of the parameter is determined before it is passed to the function." / "Call-by-Name: the value of the parameter is not determined until it is called within the function."

Question 13: What is a primary purpose of a Scala `case class` ?

- A. They can only have one instance (singleton).
- B. They are designed to be used with pattern matching and can be constructed without using `new` .
- C. They cannot be used in collections.
- D. They are mutable by default.

Answer: B. They are designed to be used with pattern matching and can be constructed without using `new` .

Explanation: `case class` is a special type of class in Scala. The slides highlight two key features: they are "used to store and match on the contents of a class," "designed to be used with pattern matching," and "You can construct them without using new." A `singleton` is defined using the `object` keyword.

Reference: Module 04 - A Crash Course on Scala - "Case classes are used to store and match on the contents of a class." / "They are designed to be used with pattern matching." / "You can construct them without using new."

Question 14: What is a "singleton" in Scala?

- A. A class that can be inherited by other classes.
- B. A `case class` used for pattern matching.
- C. A class that can have only one instance.
- D. An immutable value, similar to `val` .

Answer: C. A class that can have only one instance.

Explanation: The slides define a "singleton" as "a class that can have only one instance." This is typically created using the `object` keyword in Scala.

Reference: Module 04 - A Crash Course on Scala - "A singleton is a class that can have only one instance."

Question 15: What does the `foldLeft` operation do?

- A. It removes the first element (the "left" element) of a collection.
- B. It takes an associative binary operator function and uses it to collapse elements from the collection, starting from the head (left).

C. It sorts the collection in ascending order.

D. It mirrors the collection, turning `List(1, 2, 3)` into `List(3, 2, 1)`.

Answer: B. It takes an associative binary operator function and uses it to collapse elements from the collection, starting from the head (left).

Explanation: `foldLeft` (and `foldRight`) "Takes an associative binary operator function and uses it to collapse elements from the collection." `foldLeft` "goes through the whole List, from head (left) to tail (right)."

Reference: Module 04 - A Crash Course on Scala - "foldRight and foldLeft" / "Takes an associative binary operator function and uses it to collapse elements from the collection." / "It goes through the whole List, from head (left) to tail (right)."

Question 16: What is a `Tuple` in Scala?

A. An immutable, fixed-number of items that can be of different types.

B. A mutable collection of items of the same type.

C. A synonym for a `List` with two elements.

D. A special type of `case class` used for pattern matching.

Answer: A. An immutable, fixed-number of items that can be of different types.

Explanation: A `Tuple` is used to group a "fixed number of items of different types together" and is "Immutable." An example would be `("hello", 1, true)`.

Reference: Module 04 - A Crash Course on Scala - "A fixed number of items of different types together" / "Immutable"

Question 17: When considering an inclusive code change that doubles the runtime, what is a key question the slides suggest a developer should ask?

A. "Is the new code faster than the old code?"

B. "Can this change be implemented using `val` instead of `var`?"

C. "What inclusivity gap does the change close (e.g., languages, formats) and who benefits?"

D. "Does this change use a higher-order function?"

Answer: C. "What inclusivity gap does the change close (e.g., languages, formats) and who benefits?"

Explanation: The slides propose a structured way to evaluate such a trade-off. The first step is to "Define the goal," which includes asking: "What inclusivity gap does the change close (languages, names, calendars, formats)?" and "Who benefits and how?".

Reference: Module 04 - A Crash Course on Scala - "Define the goal" / "What inclusivity gap does the change close (languages, names, calendars, formats)?" / "Who benefits and how?"

Question 18: According to the slides, what is a primary risk of using a Scala program trained on biased data for decisions in hiring or lending?

- A. The program will be slow and inefficient.
- B. The program will be difficult to maintain due to complex code.
- C. The program could repeat and reinforce unfair historical patterns at scale.
- D. The program will fail to compile due to type errors.

Answer: C. The program could repeat and reinforce unfair historical patterns at scale.

Explanation: The slides pose a question about how such a program might reinforce inequities. The first risk listed is that "If trained on biased data, it could repeat unfair patterns for years".

Reference: Module 04 - A Crash Course on Scala - "If trained on biased data, it could repeat unfair patterns for years."

Question 19: What impact does the deep integration of Scala with proprietary platforms have on open, community-led alternatives, according to the slides?

- A. It forces the proprietary platforms to become open-source.
- B. It encourages more community-led alternatives to develop.
- C. Big companies set the direction, and open-source projects struggle to keep users and helpers.
- D. It leads to more diverse voices and ideas shaping the language's future.

Answer: C. Big companies set the direction, and open-source projects struggle to keep users and helpers.

Explanation: The slides list several negative impacts of this integration. These include: "Big companies set the direction, not the community," "Developers may get locked into one company's tools," and "Open-source projects struggle to keep users and helpers."

Reference: Module 04 - A Crash Course on Scala - "Big companies set the direction, not the community." / "Developers may get locked into one company's tools." / "Open-source projects struggle

to keep users and helpers."

Question 20: What is the result of the following Scala code?

```
val nested = List(List(1), List(2, 3))  
val flat = nested.flatten
```

- A. `List(1, 2, 3)`
- B. `List(List(1), List(2, 3))`
- C. `List(6)`
- D. The code will fail to compile.

Answer: A. `List(1, 2, 3)`

Explanation: The `flatten` operation "collapses one level of nested structure." It takes a list of lists and concatenates them into a single, "flat" list.

Reference: Module 04 - A Crash Course on Scala - "flatten" / "It collapses one level of nested structure."

Module 05 – Parallel Processing – MapReduce (pp. 461–558)

Question 1: What is the fundamental difference between "scale up" (vertical scaling) and "scale out" (horizontal scaling)?

- A. Scale up adds more nodes to a system, while scale out adds resources to a single node.
- B. Scale up adds resources to a single node, while scale out adds more nodes to a system.
- C. Scale up is used for MapReduce, while scale out is used for traditional databases.
- D. Scale up adds resources (like CPU/RAM) to a single node, while scale out uses a shared-disk architecture.

Answer: B. Scale up adds resources to a single node, while scale out adds more nodes to a system.

Explanation: The slides define "Scale up or scale vertically: adding resources to a single node in a system." In contrast, "Scale out or scale horizontally: adding more nodes to a system," which is the basis of the "shared nothing architecture".

Reference: Module 05 - Parallel Processing - MapReduce - "Scale up or scale vertically: adding resources to a single node in a system." / "Scale out or scale horizontally: adding more nodes to a system."

Question 2: MapReduce is described as both a programming model and an execution framework. What is a key goal of this abstraction?

- A. To force programmers to manually manage data distribution and machine failures.
- B. To hide system-level details like parallelization, fault tolerance, and data distribution from the programmer.
- C. To ensure all data fits in the memory of a single machine.
- D. To replace the need for a distributed file system like HDFS.

Answer: B. To hide system-level details like parallelization, fault tolerance, and data distribution from the programmer.

Explanation: The core idea of MapReduce is to simplify distributed programming. The framework "MapReduce takes care of parallelization, fault tolerance, and data distribution," and its goal is to "Hide system-level details from programmers."

Reference: Module 05 - Parallel Processing - MapReduce - "MapReduce takes care of parallelization, fault tolerance, and data distribution." / "Hide system-level details from programmers."

Question 3: In a MapReduce job, what is the primary role of the **Map** task?

- A. To sort and consolidate all intermediate key/value pairs from all mappers.
- B. To operate on a single input split (typically one HDFS block) and generate a set of intermediate key/value pairs.
- C. To operate on all intermediate values associated with the same key and produce the final output.
- D. To assign work to idle workers and monitor for failures.

Answer: B. To operate on a single input split (typically one HDFS block) and generate a set of intermediate key/value pairs.

Explanation: "Each Map task (typically) operates on a single HDFS block." Its job is to process that input and "generates a set of intermediate key/value pairs." The Reduce task handles operating on values with the same key.

Reference: Module 05 - Parallel Processing - MapReduce - "Each Map task (typically) operates on a single HDFS block." / "Each Map task generates a set of intermediate key/value pairs."

Question 4: What is the "Shuffle" phase in MapReduce?

- A. The phase where the user program divides input files into M splits.
- B. The phase where `Map` tasks write their output to local disk.
- C. The phase where intermediate data is sorted and consolidated from all mappers before the `Reduce` tasks start.
- D. The phase where the master assigns map tasks to workers.

Answer: C. The phase where intermediate data is sorted and consolidated from all mappers before the `Reduce` tasks start.

Explanation: The Shuffle is the phase that "Sorts and consolidates intermediate data from all mappers." It "Happens after all Map tasks are complete and before Reduce tasks start."

Reference: Module 05 - Parallel Processing - MapReduce - "Sorts and consolidates intermediate data from all mappers." / "Happens after all Map tasks are complete and before Reduce tasks start."

Question 5: What is the purpose of a "Combiner" in a MapReduce job?

- A. To combine the final output files from all reducers into a single file.
- B. To merge partially aggregated data on the map-side *before* it is sent over the network to the reducers.
- C. To join two or more datasets together during the map phase.
- D. To ensure that completed reduce tasks are not re-executed upon failure.

Answer: B. To merge partially aggregated data on the map-side *before* it is sent over the network to the reducers.

Explanation: A Combiner performs "Local Aggregation." It is used when "there is significant repetition in the intermediate keys produced by each map task" to "Merge partially data before it is sent over the network to the reducer." This reduces network traffic.

Reference: Module 05 - Parallel Processing - MapReduce - "Local Aggregation" / "Merge partially data before it is sent over the network to the reducer."

Question 6: In the provided MapReduce Java code example, which class is set for both the `CombinerClass` and the `ReducerClass` ?

- A. `MyMap.class`

- B. `MyReduce.class`
- C. `TextInputFormat.class`
- D. `Job.class`

Answer: B. `MyReduce.class`

Explanation: The code example shows `job.setCombinerClass(MyReduce.class);` and `job.setReducerClass(MyReduce.class);`. This is a common pattern because the local aggregation (combining) and the final aggregation (reducing) are "Typically the same code for the combiner and the reduce function."

Reference: Module 05 - Parallel Processing - MapReduce - "`job.setCombinerClass(MyReduce.class);`" / "`job.setReducerClass(MyReduce.class);`" / "Typically the same code for the combiner and the reduce function."

Question 7: How does the MapReduce framework handle a `Map` task worker failure?

- A. The master detects the failure via heartbeats and re-executes the in-progress or completed map task on a different worker.
- B. The job fails and must be restarted manually by the user.
- C. Completed map tasks do not need to be re-executed, as their output is in a global filesystem.
- D. The master checkpoints its state and restarts from the last checkpoint.

Answer: A. The master detects the failure via heartbeats and re-executes the in-progress or completed map task on a different worker.

Explanation: Failure detection happens via "periodic heartbeats." If a map worker fails, any "in-progress map... tasks" are re-executed. Crucially, "Re-execute completed map tasks" is also necessary because "their output is stored on the local disk of the failed machine and is therefore inaccessible."

Reference: Module 05 - Parallel Processing - MapReduce - "Detect failure via periodic heartbeats." / "Re-execute completed map tasks: their output is stored on the local disk of the failed machine and is therefore inaccessible."

Question 8: Why does a completed `Reduce` task *not* need to be re-executed if its worker fails?

- A. Because its output is stored on the local disk of the failed machine.
- B. Because its output is stored in a global filesystem (like HDFS).

C. Because the master checkpoints the output of all reduce tasks.

D. Because the reduce function is commutative and associative.

Answer: B. Because its output is stored in a global filesystem (like HDFS).

Explanation: Unlike map task outputs (which are local), "Completed reduce tasks do not need to be re-executed since their output is stored in a global filesystem." Therefore, if the reduce worker fails *after* completing its task, the output is already safe.

Reference: Module 05 - Parallel Processing - MapReduce - "Completed reduce tasks do not need to be re-executed since their output is stored in a global filesystem."

Question 9: You need to join a very large dataset (UserTransactions) with a very small dataset (UserDetails) that fits in memory. Which type of join is most appropriate in MapReduce?

A. Reduce-side join (Repartition join)

B. Map-side join (Replication join)

C. A full-shuffle join.

D. This type of join is not possible in MapReduce.

Answer: B. Map-side join (Replication join)

Explanation: The slides present two join strategies. A "Reduce-side join" is for "joining two or more large datasets." A "Map-side join" (or "Replication join") is used "When one of the datasets is small enough to cache." In this scenario, the small UserDetails dataset can be replicated to all map nodes.

Reference: Module 05 - Parallel Processing - MapReduce - "Reduce-side join" / "When joining two or more large datasets together" / "Map-side join" / "Replication join" / "When one of the datasets is small enough to cache"

Question 10: In the MapReduce execution flow, where are the intermediate key/value pairs from the `map` function first buffered?

A. In the master node's memory.

B. Directly to the global filesystem (HDFS).

C. In memory on the map worker, before being periodically written to its local disk.

D. In memory on the reduce workers.

Answer: C. In memory on the map worker, before being periodically written to its local disk.

Explanation: "The key/value pairs produced by the map function are buffered in memory." Following this, "The buffered pairs are periodically written to local disk" on the map worker's machine.

Reference: Module 05 - Parallel Processing - MapReduce - "The key/value pairs produced by the map function are buffered in memory." / "The buffered pairs are periodically written to local disk."

Question 11: What is the role of the master node in the MapReduce implementation?

- A. It executes both map and reduce tasks.
- B. It reads the input splits and passes key/value pairs to the map function.
- C. It assigns map and reduce tasks to idle workers and forwards the location of intermediate data to reduce workers.
- D. It sorts the intermediate data by key before sending it to the reduce workers.

Answer: C. It assigns map and reduce tasks to idle workers and forwards the location of intermediate data to reduce workers.

Explanation: "One of the copies of the program is master". "The master assigns works to the workers" by picking "idle workers and assigns each one a map task or a reduce task." Later, "The locations of the buffered pairs... are passed back to the master," which then "forwards these locations to the reduce workers."

Reference: Module 05 - Parallel Processing - MapReduce - "One of the copies of the program is master, and the rest are workers." / "The master assigns works to the workers." / "The master forwards these locations to the reduce workers."

Question 12: How are intermediate keys partitioned for the reducers?

- A. $\text{hash}(\text{key}) \bmod R$, where R is the number of reduce tasks.
- B. $\text{hash}(\text{value}) \bmod M$, where M is the number of map tasks.
- C. They are randomly assigned to reducers to ensure load balancing.
- D. All keys are sent to a single reducer, which then partitions them.

Answer: A. $\text{hash}(\text{key}) \bmod R$, where R is the number of reduce tasks.

Explanation: When map outputs are written to local disk, "They are partitioned into R regions ($\text{hash}(\text{key}) \bmod R$)."
 R represents the number of reduce tasks. This ensures all values for the same key go to the same reducer.

Reference: Module 05 - Parallel Processing - MapReduce - "They are partitioned into R regions (hash(key) mod R)."

Question 13: When does a **Reduce** task begin?

- A. As soon as the first **Map** task completes.
- B. After all **Map** tasks are complete and the Shuffle phase has started.
- C. Concurrently with the **Map** tasks.
- D. After the master node has sorted all intermediate data.

Answer: B. After all **Map** tasks are complete and the Shuffle phase has started.

Explanation: The Shuffle phase, which "happens after all Map tasks are complete and before Reduce tasks start," is the prerequisite for the reduce phase. A reduce worker can only begin its work of sorting and reducing after the map phase is finished and it can start pulling data.

Reference: Module 05 - Parallel Processing - MapReduce - "Shuffle" / "Happens after all Map tasks are complete and before Reduce tasks start."

Question 14: What is "data skew" in the context of MapReduce?

- A. When the MapReduce framework reinforces big tech dominance.
- B. When data is forced into key-value pairs, stripping its original context.
- C. When one key has far more records than others, causing that key to dominate the job and potentially slow down a reducer.
- D. When all map tasks run on the same node, violating data locality.

Answer: C. When one key has far more records than others, causing that key to dominate the job and potentially slow down a reducer.

Explanation: Data skew is a common problem where "One key with far more records dominates the job". This can lead to "Delayed tasks: hot keys slow down reducers," creating a bottleneck.

Reference: Module 05 - Parallel Processing - MapReduce - "One key with far more records dominates the job -> data skew." / "Delayed tasks: hot keys slow down reducers, jobs may favor majority data."

Question 15: According to the slides, how can "data skew" lead to biased or unrepresentative results?

- A. It forces all data into key-value pairs, erasing context.

B. Skew and volume-based weighting can cause majority data to dominate, systematically underrepresenting minority patterns.

C. It adds system complexity, making the code harder to debug.

D. It requires using stratified sampling, which can introduce sampling bias.

Answer: B. Skew and volume-based weighting can cause majority data to dominate, systematically underrepresenting minority patterns.

Explanation: The critique of MapReduce links data skew to bias: "Bias toward data-rich groups". "Skew + volume-based weighting means majority data dominates: minority patterns are systematically underrepresented." This is also described as "Overrepresentation: large groups outweigh smaller ones in results."

Reference: Module 05 - Parallel Processing - MapReduce - "Bias toward data-rich groups" / "Skew + volume-based weighting means majority data dominates: minority patterns are systematically underrepresented." / "Overrepresentation: large groups outweigh smaller ones in results."

Question 16: Which of the following is proposed as a method to *address* data skew and its associated biases?

A. Using weighted aggregation in the reducer so small groups are not drowned out.

B. Forcing all data into key-value pairs to normalize it.

C. Ensuring all map tasks run on the node where the block is stored.

D. Designing the framework specifically for Google-scale workloads.

Answer: A. Using weighted aggregation in the reducer so small groups are not drowned out.

Explanation: The slides suggest several methods to "address them" (biases from skew). One is to modify the "Reducer logic" to "Apply weighted aggregation so small groups are not drowned out by majority counts." Other methods include stratified sampling and key splitting.

Reference: Module 05 - Parallel Processing - MapReduce - "How to address them?" / "Reducer logic" / "Apply weighted aggregation so small groups are not drowned out by majority counts."

Question 17: What is a *negative trade-off* of using weighted aggregation to mitigate skew?

A. It adds system complexity and can increase latency.

B. It may distort the results if the goal is to reflect true frequency, and it could be misused to hide actual distributions.

C. It can introduce sampling bias if strata are not chosen carefully.

D. It slows down majority group processing.

Answer: B. It may distort the results if the goal is to reflect true frequency, and it could be misused to hide actual distributions.

Explanation: The slides list "Possible negative impacts of skew mitigation?". For "Weighted aggregation," the risk is that it "May distort results if the goal is to reflect true frequency (over-correcting minority data)" and "Could be misused to justify fairness while hiding actual distributions."

Reference: Module 05 - Parallel Processing - MapReduce - "Possible negative impacts of skew mitigation?" / "Weighted aggregation" / "May distort results if the goal is to reflect true frequency (over-correcting minority data)." / "Could be misused to justify fairness while hiding actual distributions."

Question 18: What is the "erasure of context" critique of MapReduce?

A. That processing large data sets reinforces the dominance of big tech.

B. That forcing all data into key-value pairs strips nuance from social or relational data, making some knowledge systems uncomputable.

C. That data skew causes minority patterns to be erased from the results.

D. That completed map tasks are erased on failure and must be re-executed.

Answer: B. That forcing all data into key-value pairs strips nuance from social or relational data, making some knowledge systems uncomputable.

Explanation: This is one of the ethical limitations discussed. The critique is that "Forcing all data into key-value pairs strips nuance from social, cultural, or relational data: certain knowledge systems become uncomputable."

Reference: Module 05 - Parallel Processing - MapReduce - "Erasure of context" / "Forcing all data into key-value pairs strips nuance from social, cultural, or relational data: certain knowledge systems become uncomputable."

Question 19: What is a "reduce-side join"?

A. A join where one dataset is small enough to be cached on the map side.

B. A join where both datasets are large and must be repartitioned and shuffled to the reducers.

C. An operation that combines intermediate map outputs before the shuffle.

D. A join performed by the master node after all reduce tasks are complete.

Answer: B. A join where both datasets are large and must be repartitioned and shuffled to the reducers.

Explanation: A "Reduce-side join," also called a "Repartition join," is the strategy used "When joining two or more large datasets together." This requires both datasets to be processed by mappers and then "shuffled" to the reducers.

Reference: Module 05 - Parallel Processing - MapReduce - "Reduce-side join" / "Repartition join" / "When joining two or more large datasets together"

Question 20: How does the MapReduce master handle its own failure?

- A. The master cannot fail; it is a fully replicated, fault-tolerant process.
- B. If the master fails, the entire job fails and must be restarted.
- C. The master's state is periodically checkpointed, and a new master can start from the last checkpoint.
- D. A standby master immediately takes over all in-progress tasks.

Answer: C. The master's state is periodically checkpointed, and a new master can start from the last checkpoint.

Explanation: The master is a potential single point of failure. The strategy described to handle this is: "State is periodically checkpointed: a new copy of master starts from the last checkpoint state."

Reference: Module 05 - Parallel Processing - MapReduce - "State is periodically checkpointed: a new copy of master starts from the last checkpoint state."

Module 06 – Parallel Processing - Spark (pp. 559–648)

Question 1: What is a primary motivation for using Apache Spark instead of MapReduce?

- A. MapReduce is expensive (slow) because it "always goes to disk and HDFS" for intermediate data, while Spark processes data in memory.
- B. MapReduce requires data to be described as a Directed Acyclic Graph (DAG).
- C. Spark does not support fault tolerance, making it faster.
- D. Spark uses a "shared-nothing" architecture, while MapReduce uses a "shared-disk" architecture.

Answer: A. MapReduce is expensive (slow) because it "always goes to disk and HDFS" for intermediate data, while Spark processes data in memory.

Explanation: The slides point out that MapReduce's "Acyclic data flow from stable storage to stable storage" is a limitation. It is "expensive (slow), i.e., always goes to disk and HDFS". Spark was designed to

overcome this by processing data in-memory using RDDs.

Reference: Module 06 - Parallel Processing - Spark - "Acyclic data flow from stable storage to stable storage." / "MapReduce is expensive (slow), i.e., always goes to disk and HDFS."

Question 2: In a Spark application, what is the role of the **Driver** process?

- A. It executes the code assigned to it by the executors.
- B. It runs the `main()` function and is responsible for analyzing, distributing, and scheduling work across the executors.
- C. It stores all cached RDD partitions in its local memory.
- D. It is the entry point for low-level API functionality, accessed via `spark.sparkContext`.

Answer: B. It runs the `main()` function and is responsible for analyzing, distributing, and scheduling work across the executors.

Explanation: The Driver is the "heart of a Spark application". It "Runs the `main()` function" and is "Responsible for three things:", including "Analyzing, distributing, and scheduling work across the executors". Executors, in turn, "Executing code assigned to it by the driver".

Reference: Module 06 - Parallel Processing - Spark - "Runs the `main()` function" / "Responsible for three things:" / "Analyzing, distributing, and scheduling work across the executors" / "Executing code assigned to it by the driver"

Question 3: What is a Resilient Distributed Dataset (RDD) in Spark?

- A. A mutable collection of objects spread across a cluster.
- B. An immutable, distributed memory abstraction.
- C. A table-like collection with well-defined rows and columns.
- D. A data structure that is only stored on the driver node.

Answer: B. An immutable, distributed memory abstraction.

Explanation: An RDD is a "distributed memory abstraction". It is an "Immutable collections of objects spread across a cluster". RDDs are divided into partitions that can be stored on different nodes.

Reference: Module 06 - Parallel Processing - Spark - "A distributed memory abstraction." / "Immutable collections of objects spread across a cluster."

Question 4: What are the two main types of RDD operations?

- A. `read` and `write`
- B. `cache` and `checkpoint`
- C. `narrow` and `wide`
- D. `Transformations` and `Actions`

Answer: D. `Transformations` and `Actions` .

Explanation: RDDs support "two types of operations": "Transformations: allow us to build the logical plan" and "Actions: allow us to trigger the computation".

Reference: Module 06 - Parallel Processing - Spark - "RDDs support two types of operations:" / "Transformations: allow us to build the logical plan" / "Actions: allow us to trigger the computation"

Question 5: What does it mean for Spark transformations to be "lazy"?

- A. They are not computed immediately; they are only computed when an action requires a result.
- B. They are computationally inefficient and should be avoided.
- C. They must be explicitly started by calling a `.run()` method.
- D. They can only be applied to RDDs that are cached in memory.

Answer: A. They are not computed immediately; they are only computed when an action requires a result.

Explanation: All "transformations are lazy." This means Spark does "Not compute their results right away." Instead, it "Remember[s] the transformations applied". The computation is "only computed when an action requires a result to be returned to the driver program".

Reference: Module 06 - Parallel Processing - Spark - "All transformations are lazy." / "Not compute their results right away." / "They are only computed when an action requires a result to be returned to the driver program."

Question 6: Which of the following is an **Action** in Spark?

- A. `map`
- B. `reduceByKey`
- C. `join`
- D. `collect`

Answer: D. `collect`

Explanation: `map`, `reduceByKey`, and `join` are all transformations that create a new RDD.

`collect` is an action that triggers the computation and "returns all the elements of the RDD as an array at the driver."

Reference: Module 06 - Parallel Processing - Spark - "collect returns all the elements of the RDD as an array at the driver." / "map applies a given function..." / "reduceByKey... Aggregate the values..." / "join performs an inner-join..."

Question 7: What is the RDD "lineage graph"?

- A. A graph showing the dependencies between executors.
- B. A DAG (Directed Acyclic Graph) representing the computations (transformations) used to build an RDD.
- C. A log of which users have accessed a particular RDD.
- D. A graph representing the physical location of RDD partitions.

Answer: B. A DAG (Directed Acyclic Graph) representing the computations (transformations) used to build an RDD.

Explanation: "Lineage: transformations used to build an RDD." "RDDs are stored as a chain of objects capturing the lineage of each RDD." This is also described as a "DAG representing the computations done on the RDD".

Reference: Module 06 - Parallel Processing - Spark - "Lineage: transformations used to build an RDD." / "A DAG representing the computations done on the RDD is called lineage graph."

Question 8: How does Spark use the lineage graph for fault tolerance?

- A. It replicates all RDD partitions to at least three different nodes.
- B. It checkpoints every RDD to disk after every transformation.
- C. If a partition is lost (e.g., an executor fails), Spark recomputes only the lost partition using the lineage graph.
- D. It saves the entire RDD to disk using `saveAsTextFile` after each action.

Answer: C. If a partition is lost (e.g., an executor fails), Spark recomputes only the lost partition using the lineage graph.

Explanation: Spark does "No replication." Instead, "Lineages are the key to fault tolerance in Spark." The system can "Recompute only the lost partitions of an RDD" by following the lineage graph.

Reference: Module 06 - Parallel Processing - Spark - "No replication." / "Lineages are the key to fault tolerance in Spark." / "Recompute only the lost partitions of an RDD."

Question 9: What is the difference between a "narrow" transformation and a "wide" transformation?

- A. Narrow transformations are lazy, while wide transformations are eager.
- B. Narrow transformations involve each *input* partition contributing to only *one* output partition, while wide transformations involve an input partition contributing to *many* output partitions.
- C. Narrow transformations are actions, while wide transformations are transformations.
- D. Narrow transformations require a `reduceByKey` operation, while wide transformations require a `map` operation.

Answer: B. Narrow transformations involve each *input* partition contributing to only *one* output partition, while wide transformations involve an input partition contributing to *many* output partitions.

Explanation: In a "Narrow transformations (dependencies)", "Each input partition will contribute to only one output partition." In a "Wide transformations (dependencies)", "Each input partition will contribute to many output partition." Wide transformations are "Usually referred to as a shuffle".

Reference: Module 06 - Parallel Processing - Spark - "Narrow transformations (dependencies)" / "Each input partition will contribute to only one output partition." / "Wide transformations (dependencies)" / "Each input partition will contribute to many output partition."

Question 10: In Spark's execution hierarchy, what defines the boundary between "stages"?

- A. Actions
- B. Narrow transformations
- C. Wide transformations (shuffles)
- D. The `cache()` operation

Answer: C. Wide transformations (shuffles)

Explanation: A Spark job (triggered by an action) is broken down into stages. "Wide transformations define the breakdown of jobs into stages." A stage is a group of tasks that can be executed together without a shuffle.

Reference: Module 06 - Parallel Processing - Spark - "Wide transformations define the breakdown of jobs into stages."

Question 11: What is the purpose of the `cache()` operation on an RDD?

- A. To save the RDD to disk so it is not lost when the SparkContext is destroyed.
- B. To instruct Spark to store the RDD's partitions in memory on the nodes that compute them, so they can be reused in future actions.
- C. To trigger the computation of all lazy transformations.
- D. To convert a generic RDD into a key-value RDD.

Answer: B. To instruct Spark to store the RDD's partitions in memory on the nodes that compute them, so they can be reused in future actions.

Explanation: "When you cache an RDD, each node stores any partitions of it that it computes in memory." This is an optimization. "An RDD that is not cached is re-evaluated each time an action is invoked". If it is cached, a "node reuses the cached RDD in other actions on that dataset."

Reference: Module 06 - Parallel Processing - Spark - "When you cache an RDD, each node stores any partitions of it that it computes in memory." / "An RDD that is not cached is re-evaluated each time an action is invoked on that RDD." / "A node reuses the cached RDD in other actions on that dataset."

Question 12: What is the difference between `cache()` and `checkpoint()` ?

- A. `cache()` stores the RDD in memory, while `checkpoint()` saves the RDD to disk and severs its lineage.
- B. `cache()` is a transformation, while `checkpoint()` is an action.
- C. `cache()` saves to disk, while `persist(level)` saves to memory.
- D. `cache()` is fault-tolerant, while `checkpoint()` is not.

Answer: A. `cache()` stores the RDD in memory, while `checkpoint()` saves the RDD to disk and severs its lineage.

Explanation: `cache` "caches the RDD into memory". `checkpoint` "saves an RDD to disk". A critical difference is that "When we reference a checkpointed RDD, it will derive from the checkpoint instead of the source data", effectively cutting its dependency on the original lineage.

Reference: Module 06 - Parallel Processing - Spark - "cache caches the RDD into memory" / "checkpoint saves an RDD to disk." / "When we reference a checkpointed RDD, it will derive from the checkpoint

instead of the source data."

Question 13: Given `val rdd = sc.parallelize(List("a b", "c d"))`, what is the result of `rdd.map(line => line.split(" ")).collect()` ?

- A. `Array("a", "b", "c", "d")`
- B. `Array(Array("a", "b"), Array("c", "d"))`
- C. `Array(("a", 1), ("b", 1), ("c", 1), ("d", 1))`
- D. This code will not compile.

Answer: B. `Array(Array("a", "b"), Array("c", "d"))`

Explanation: The `map` transformation "applies a given function on each RDD record independently". The RDD has two elements: `"a b"` and `"c d"`. The `split(" ")` function transforms `"a b"` into `Array("a", "b")` and `"c d"` into `Array("c", "d")`. The `collect` action returns these results as an array. To get `Array("a", "b", "c", "d")`, you would need to use `flatMap`.

Reference: Module 06 - Parallel Processing - Spark - "map applies a given function on each RDD record independently." / "collect returns all the elements of the RDD as an array at the driver."

Question 14: Which operation is required to create a "key-value RDD" from a generic RDD?

- A. A `map` transformation that returns a tuple (key, value) structure.
- B. A `collect` action.
- C. A `cache` operation.
- D. A `parallelize` operation.

Answer: A. A `map` transformation that returns a tuple (key, value) structure.

Explanation: A generic RDD holds a collection of any object. To make a key-value RDD, which has "special operations, such as aggregation", you must first "map over your current RDD to a basic key-value structure", typically a tuple `(k, v)`.

Reference: Module 06 - Parallel Processing - Spark - "To make a key-value RDD:" / "map over your current RDD to a basic key-value structure." / "Key-value RDDs have special operations, such as aggregation"

Question 15: What is the purpose of the `reduce` action?

- A. To filter an RDD based on a predicate.
- B. To return all elements of the RDD to the driver.
- C. To aggregate the elements of an RDD using a given function that must be commutative and associative.
- D. To save the RDD to a text file.

Answer: C. To aggregate the elements of an RDD using a given function that must be commutative and associative.

Explanation: The `reduce` action "aggregates the elements of the dataset using a given function." This function "should be commutative and associative so that it can be computed correctly in parallel."

Reference: Module 06 - Parallel Processing - Spark - "reduce aggregates the elements of the dataset using a given function." / "The given function should be commutative and associative so that it can be computed correctly in parallel."

Question 16: In the Spark execution hierarchy, what is the "smallest execution unit"?

- A. A Job
- B. A Stage
- C. A Task
- D. An RDD

Answer: C. A Task

Explanation: The hierarchy is Job -> Stage -> Task. "A stage consists of tasks, which are the smallest execution unit." Each task "represents one local computation".

Reference: Module 06 - Parallel Processing - Spark - "A stage consists of tasks, which are the smallest execution unit." / "Each task represents one local computation."

Question 17: Prior to Spark 2.0, why did separate contexts need to be created?

- A. Because `SparkSession` did not exist, and separate `SparkContexts` were needed to use the APIs for SQL, Hive, and streaming.
- B. To connect to different clusters simultaneously.
- C. To separate narrow transformations from wide transformations.
- D. To ensure fault tolerance by replicating the driver.

Answer: A. Because `SparkSession` did not exist, and separate `SparkContexts` were needed to use the APIs for SQL, Hive, and streaming.

Explanation: "Prior to Spark 2.0.0, a the spark driver program uses `SparkContext` to connect to the cluster." "In order to use APIs of SQL, Hive and streaming, separate `SparkContexts` should to be created." `SparkSession` was introduced to provide a unified entry point.

Reference: Module 06 - Parallel Processing - Spark - "Prior to Spark 2.0.0..." / "In order to use APIs of SQL, Hive and streaming, separate `SparkContexts` should to be created."

Question 18: Which of the following is a "key-value RDD" transformation?

- A. `map`
- B. `filter`
- C. `reduceByKey`
- D. `collect`

Answer: C. `reduceByKey`

Explanation: Key-value RDDs have "special operations, such as aggregation". `reduceByKey` is a transformation (not an action like `reduce`) that "Aggregate[s] the values associated with each key" and returns a new RDD. `map` and `filter` are generic transformations. `collect` is an action.

Reference: Module 06 - Parallel Processing - Spark - "Key-value RDDs have special operations, such as aggregation" / "reduceByKey: Aggregate the values associated with each key."

Question 19: According to the slides, what is a risk of using Spark for public decision-making (e.g., in healthcare or housing)?

- A. The RDD abstraction is not suitable for structured data.
- B. The lineage graph can become too complex, leading to slow fault recovery.
- C. If input data reflects past discrimination, Spark will learn and repeat those biased patterns at scale.
- D. The `persist()` operation is not secure for sensitive public data.

Answer: C. If input data reflects past discrimination, Spark will learn and repeat those biased patterns at scale.

Explanation: A key risk discussed is "Biased inputs -> biased outputs". "If the input data reflects past discrimination (e.g., in jobs or housing), Spark will repeat those patterns at scale". This can make "Unfair

results... look objective".

Reference: Module 06 - Parallel Processing - Spark - "Biased inputs -> biased outputs" / "If the input data reflects past discrimination (e.g., in jobs or housing), Spark will repeat those patterns at scale."

Question 20: What is Dask, and why is it presented as a "feminist Spark alternative"?

- A. It is a more scalable version of Spark developed at Google.
- B. It is a Python-native distributed system that is "more accessible than Spark" and can run on laptops or clusters.
- C. It is an open standard for tracking pipeline provenance.
- D. It is a system that automatically removes bias from input data.

Answer: B. It is a Python-native distributed system that is "more accessible than Spark" and can run on laptops or clusters.

Explanation: Dask is presented as a "Lightweight and community-friendly" alternative. It is described as a "Dask: Python-native distributed system, runs on laptops or clusters, more accessible than Spark." This accessibility aligns with the critique of large, resource-heavy systems.

Reference: Module 06 - Parallel Processing - Spark - "Feminist Spark alternatives?" / "Lightweight and community-friendly" / "Dask: Python-native distributed system, runs on laptops or clusters, more accessible than Spark."

Module 07 – Structured Data Processing - Spark SQL (pp. 649–749)

Question 1: What key piece of information do RDDs lack that Spark's structured collections (DataFrames/Datasets) possess?

- A. Data
- B. Partitions
- C. Schema (information about column names and types)
- D. Lazy evaluation

Answer: C. Schema (information about column names and types)

Explanation: The slides use an `RDD[Account]` as an example. While the `Account` case class has a schema, the "RDDs don't know anything about the schema of the data it's dealing with". In contrast, a database or a DataFrame *does* see the data as "columns of named and typed values", which is the schema.

Reference: Module 07 - Structured Data Processing - "RDDs don't know anything about the schema of the data it's dealing with." / "A database/Hive sees it as a columns of named and typed values."

Question 2: What is the relationship between a DataFrame and a Dataset in Spark?

- A. A DataFrame is a "typed" collection, while a Dataset is an "untyped" collection.
- B. A DataFrame is an alias for `Dataset[Row]`, where `Row` is a generic, untyped JVM object.
- C. A DataFrame is an RDD, while a Dataset is a table.
- D. DataFrames are eagerly evaluated, while Datasets are lazily evaluated.

Answer: B. A DataFrame is an alias for `Dataset[Row]`, where `Row` is a generic, untyped JVM object.

Explanation: The "Dataset API unifies the DataFrame and RDD APIs." "You can consider a DataFrame as an alias for `Dataset[Row]`, where a `Row` is a generic untyped JVM object." A Dataset, by contrast, is a "typed distributed collections of data".

Reference: Module 07 - Structured Data Processing - "Dataset API unifies the DataFrame and RDD APIs." / "You can consider a DataFrame as an alias for `Dataset[Row]`, where a `Row` is a generic untyped JVM object." / "Datasets can be thought of as typed distributed collections of data."

Question 3: Why does the following code compile but produce a *runtime* exception?

```
val df = ... // DataFrame with columns [name, age, id]
df.filter($"id num" > 10).show()
```

- A. Because `show()` is an invalid action.
- B. Because DataFrames are "untyped" (`Dataset[Row]`), and the Scala compiler cannot type-check the schema to see that the column "id num" does not exist.
- C. Because `filter` is not a valid transformation.
- D. Because RDDs do not know the schema.

Answer: B. Because DataFrames are "untyped" (`Dataset[Row]`), and the Scala compiler cannot type-check the schema to see that the column "id num" does not exist.

Explanation: This is the key problem with DataFrames that Datasets solve. A DataFrame's elements are "Rows, which are generic untyped JVM objects." "Scala compiler cannot type check Spark SQL schemas in DataFrames." Therefore, a typo like `"id num" is not in the DataFrame columns` "compiles, but you get a runtime exception."

Reference: Module 07 - Structured Data Processing - "DataFrames elements are Rows, which are generic untyped JVM objects." / "Scala compiler cannot type check Spark SQL schemas in DataFrames." / "The following code compiles, but you get a runtime exception." / "id num is not in the DataFrame columns [name, age, id]"

Question 4: How can you convert a DataFrame `df` (which you know has the correct structure) into a typed `Dataset[Account]` ?

- A. `df.toDS()`
- B. `df.as[Account]`
- C. `df.collect()`
- D. This conversion is not possible.

Answer: B. `df.as[Account]`

Explanation: The slides show two ways to create a Dataset. `toDS()` can convert a sequence or RDD. To convert an existing DataFrame (which is a `Dataset[Row]`) to a typed Dataset, "You can call `as[SomeCaseClass]` to convert the DataFrame to a Dataset."

Reference: Module 07 - Structured Data Processing - "You can call `as[SomeCaseClass]` to convert the DataFrame to a Dataset."

Question 5: What is the purpose of `spark.sql("...")` ?

- A. To convert a DataFrame into an RDD.
- B. To run SQL queries on tables or views registered in the `SparkSession`.
- C. To define a new case class for a Dataset.
- D. To create a new `SparkSession`.

Answer: B. To run SQL queries on tables or views registered in the `SparkSession`.

Explanation: "You can run SQL queries on views/tables via the method `sql` on the `SparkSession` object." These views are typically created using `createOrReplaceTempView`.

Reference: Module 07 - Structured Data Processing - "You can run SQL queries on views/tables via the method `sql` on the `SparkSession` object."

Question 6: Which `DataFrame` transformation is used to add a new column?

- A. `drop`
- B. `select`
- C. `withColumn`
- D. `withColumnRenamed`

Answer: C. `withColumn`

Explanation: The `withColumn` transformation is used to "add[] a new column to a `DataFrame`".
`withColumnRenamed` renames an existing column, and `drop` removes one.

Reference: Module 07 - Structured Data Processing - "withColumn adds a new column to a `DataFrame`."

Question 7: What is the difference between a "shuffle join" and a "broadcast join"?

- A. A shuffle join is for small tables, and a broadcast join is for large tables.
- B. A shuffle join is an inner join, and a broadcast join is an outer join.
- C. A shuffle join involves all nodes talking to each other (shuffle), while a broadcast join sends one small table to all worker nodes to join with a big table.
- D. A shuffle join is a transformation, while a broadcast join is an action.

Answer: C. A shuffle join involves all nodes talking to each other (shuffle), while a broadcast join sends one small table to all worker nodes to join with a big table.

Explanation: The slides describe "Two different communication ways during joins". A "Shuffle join" is for "big table to big table" and requires nodes to "talk to every other node". A "Broadcast join" is for "big table to small table" and is used "When the table is small enough to fit into the memory of a single worker node."

Reference: Module 07 - Structured Data Processing - "Shuffle join: big table to big table" / "Broadcast join: big table to small table" / "Every node talks to every other node." / "When the table is small enough to fit into the memory of a single worker node."

Question 8: What is a "Window" function in Spark SQL?

- A. An aggregation (like `avg`) computed over a specific "window" (group) of rows, where each row can fall into one or more frames.
- B. A function that opens a new browser window to display query results.
- C. A `groupBy` operation that groups the entire DataFrame into one window.
- D. A function to filter rows based on a time window.

Answer: A. An aggregation (like `avg`) computed over a specific "window" (group) of rows, where each row can fall into one or more frames.

Explanation: A window function allows you to "Comput[e] some aggregation on a specific window of data." The "window determines which rows will be passed in to this function." The key difference from a `groupBy` is that "unlike grouping, here each row can fall into one or more frames".

Reference: Module 07 - Structured Data Processing - "Computing some aggregation on a specific window of data." / "The window determines which rows will be passed in to this function." / "Unlike grouping, here each row can fall into one or more frames."

Question 9: How do you define a window specification in Spark that includes the current row, one row before, and one row after?

- A. `Window.partitionBy("name")`
- B. `Window.rowsBetween(-1, 1)`
- C. `Window.groupBy("name")`
- D. `avg(col("age")).over(-1, 1)`

Answer: B. `Window.rowsBetween(-1, 1)`

Explanation: The code example explicitly defines this window: `val windowSpec = Window.rowsBetween(-1, 1)`. This specification is then applied to an aggregation function using `.over(windowSpec)`.

Reference: Module 07 - Structured Data Processing - "val windowSpec = Window.rowsBetween(-1, 1)"

Question 10: Which of the following is *not* listed as a supported raw data source for DataFrames?

- A. CSV, JSON, Parquet, ORC
- B. Cassandra, HBase, MongoDB

C. Plain-text files

D. `RDD[Account]` objects

Answer: D. `RDD[Account]` objects

Explanation: The slides list "Two ways to create a DataFrame: 1. From an RDD 2. From raw data sources". The list of *data sources* includes "CSV, JSON, Parquet, ORC" and databases like "Cassandra, HBase, MongoDB". An RDD is a different *origin*, not a "raw data source" in the same category as a file format.

Reference: Module 07 - Structured Data Processing - "Two ways to create a DataFrame:" / "1. From an RDD" / "2. From raw data sources" / "CSV, JSON, Parquet, ORC..." / "Cassandra, HBase, MongoDB..."

Question 11: What is a DataFrame `Row` ?

A. A strongly-typed case class instance.

B. A record of data that has a schema.

C. A record of data, of type `Row` , which does *not* have a schema, requiring positional access.

D. A reference to a column in a DataFrame.

Answer: C. A record of data, of type `Row` , which does *not* have a schema, requiring positional access.

Explanation: A `Row` is a "record of data". However, the slides explicitly state that "Rows do not have schemas." To access data within them, "you need to specify the position that you would like." This is the "untyped" nature of DataFrames.

Reference: Module 07 - Structured Data Processing - "A row is a record of data." / "They are of type Row." / "Rows do not have schemas." / "To access data in rows, you need to specify the position that you would like."

Question 12: Which operation is used to define a new column-based function?

A. `udf` (User-Defined Function)

B. `withColumn`

C. `selectExpr`

D. `as[CaseClass]`

Answer: A. `udf` (User-Defined Function)

Explanation: The slides state, "You can use udf to define new column-based functions." This allows you to register a Scala function so it can be used in DataFrame/SQL operations.

Reference: Module 07 - Structured Data Processing - "You can use udf to define new column-based functions."

Question 13: Which DataFrame *action* displays the top 20 rows in a tabular format?

- A. `collect`
- B. `take(20)`
- C. `head`
- D. `show`

Answer: D. `show`

Explanation: `show` is an action that "displays the top 20 rows of the DataFrame in a tabular form".
`collect` "returns an array that contains all of rows", and `take(n)` "returns the first n rows".

Reference: Module 07 - Structured Data Processing - "show: displays the top 20 rows of the DataFrame in a tabular form."

Question 14: In what two phases is a `groupBy` aggregation performed?

- A. 1. Specify the window; 2. Specify the aggregation.
- B. 1. Specify the column(s) to group on; 2. Specify the aggregation(s) using `.agg`.
- C. 1. Use `select` to choose columns; 2. Use `filter` to group them.
- D. 1. Use `udf` to define the group; 2. Use `map` to perform the aggregation.

Answer: B. 1. Specify the column(s) to group on; 2. Specify the aggregation(s) using `.agg`.

Explanation: The slides explicitly state that grouping is done in "two phases: 1. Specify the column(s) on which we would like to group. 2. Specify the aggregation(s) using the agg operation."

Reference: Module 07 - Structured Data Processing - "We do this grouping in two phases:" / "1. Specify the column(s) on which we would like to group." / "2. Specify the aggregation(s) using the agg operation."

Question 15: Which of the following is an "Outer" join?

- A. A join that keeps rows that have a match in both DataFrames.

- B. A join that keeps rows that have a match in the left DataFrame but not the right.
- C. A join that keeps rows from both sides, regardless of whether they have a match, inserting `null` s where data is missing.
- D. A join that returns all possible combinations of rows from both tables (Cartesian product).

Answer: C. A join that keeps rows from both sides, regardless of whether they have a match, inserting `null` s where data is missing.

Explanation: The `outer join` is one of the listed join types. The example output for an outer join clearly shows all rows from both tables are preserved. Rows that do not have a match in the other table (like 'Sarunas' and 'Seif' for department, or 'SICS' for person) have `null` values inserted in the corresponding columns.

Reference: Module 07 - Structured Data Processing - "Different join types: inner join, outer join, left outer join, right outer join..." / (Visual Example of Outer Join)

Question 16: How do Datasets provide "type safety" that DataFrames lack?

- A. By running all code on the driver node.
- B. By using `Dataset[Account]` instead of `Dataset[Row]` , allowing the Scala compiler to check column names and types at compile-time.
- C. By forcing all columns to be of type `String` .
- D. By automatically casting types at runtime to avoid exceptions.

Answer: B. By using `Dataset[Account]` instead of `Dataset[Row]` , allowing the Scala compiler to check column names and types at compile-time.

Explanation: The problem with DataFrames (`Dataset[Row]`) is the "Scala compiler cannot type check Spark SQL schemas in DataFrames," leading to runtime exceptions. Datasets are "typed distributed collections of data". By using a typed `Dataset[Account]` , an error like `account.id_num` (instead of `account.id`) would be caught by the *compiler*.

Reference: Module 07 - Structured Data Processing - "Scala compiler cannot type check Spark SQL schemas in DataFrames." / "Wouldn't it be nice if we could have both Spark SQL optimizations and typesafety?" / "Datasets can be thought of as typed distributed collections of data."

Question 17: What is a "schema" in the context of a DataFrame?

- A. The RDD lineage graph.

B. The list of column names and their data types.

C. A user-defined function (`udf`).

D. The physical location of the data partitions.

Answer: B. The list of column names and their data types.

Explanation: A schema "Defines the column names and types of a DataFrame". The example shows `people.json` being read and its schema (age: long, name: string) being inferred.

Reference: Module 07 - Structured Data Processing - "Defines the column names and types of a DataFrame."

Question 18: According to the slides, what is a primary risk of forcing complex identities (e.g., gender, race) into rigid schemas?

A. It improves query performance but uses more storage.

B. These identities get "flattened into rigid boxes," and values that don't fit the schema may be deleted or misclassified.

C. It makes the system reflect the data subjects' worldview, not the designers'.

D. It requires using probabilistic databases like MystiQ.

Answer: B. These identities get "flattened into rigid boxes," and values that don't fit the schema may be deleted or misclassified.

Explanation: The slides critique strict schemas, stating that "Complex identities (e.g., gender, race, culture) get flattened into rigid boxes." A direct risk is that "Values that don't fit the schema may be deleted or misclassified."

Reference: Module 07 - Structured Data Processing - "What are the risks of forcing data into strict structures..." / "Complex identities (e.g., gender, race, culture) get flattened into rigid boxes." / "Values that don't fit the schema may be deleted or misclassified."

Question 19: What is a critique of "filling in" missing data with defaults (e.g., 0, "unknown")?

A. It makes the dataset too large.

B. It can create a false sense of completeness and hide the structural inequalities that caused the missingness.

C. It is not supported by DataFrames, only Datasets.

D. It requires using a document-oriented NoSQL database like MongoDB.

Answer: B. It can create a false sense of completeness and hide the structural inequalities that caused the missingness.

Explanation: When missing data is handled, "Filling in with defaults (0, "unknown") can create a false sense of completeness." This is a problem because the "Structural inequalities that caused the missingness get hidden."

Reference: Module 07 - Structured Data Processing - "What happens when missing data is dropped or filled in?" / "Filling in with defaults (0, "unknown") can create a false sense of completeness." / "Structural inequalities that caused the missingness get hidden."

Question 20: Which alternative database model is proposed as a way to *avoid* the problems of strict, rigid schemas?

A. Relational databases with strict normalization.

B. Document-oriented NoSQL databases (like MongoDB) because they allow semi-structured data where different documents can have different fields.

C. Key-value stores.

D. Probabilistic databases like MayBMS.

Answer: B. Document-oriented NoSQL databases (like MongoDB) because they allow semi-structured data where different documents can have different fields.

Explanation: As an alternative to rigid schemas, the slides suggest "Document-oriented NoSQL DBs (e.g., MongoDB, CouchDB)". The reason is that "They allow semi-structured data (different documents can have different fields), so not everything needs to fit a fixed schema."

Reference: Module 07 - Structured Data Processing - "Document-oriented NoSQL DBs (e.g., MongoDB, CouchDB)" / "They allow semi-structured data (different documents can have different fields), so not everything needs to fit a fixed schema."

Module 08 – Data Stream Processing (pp. 750–851)

Question 1: What is the fundamental characteristic of input data in stream processing?

A. The data is bounded, with a predetermined beginning and end.

B. The data consists only of structured tuples.

C. The input data is unbounded, representing a series of events with no predetermined beginning or end.

D. The data must arrive in perfect time order.

Answer: C. The input data is unbounded, representing a series of events with no predetermined beginning or end.

Explanation: Stream processing involves "real-time computation of continuously incoming data." The defining characteristic is that "The input data is unbounded: a series of events, no predetermined beginning or end."

Reference: Module 08 - Data Stream Processing - "Stream processing is the real-time computation of continuously incoming data." / "The input data is unbounded: a series of events, no predetermined beginning or end."

Question 2: What is the difference between "event time" and "processing time" in stream processing?

A. Event time is when a record is received by the application; processing time is when the event actually occurred.

B. Event time is when the event actually occurred (timestamped at the source); processing time is when the record is received by the streaming application.

C. Event time is used for micro-batch systems; processing time is used for continuous processing systems.

D. There is no difference; the terms are interchangeable.

Answer: B. Event time is when the event actually occurred (timestamped at the source); processing time is when the record is received by the streaming application.

Explanation: "Event time: the time at which events actually occurred," typically using "Timestamps inserted into each record at the source." "Processing time: the time when the record is received at the streaming application."

Reference: Module 08 - Data Stream Processing - "Event time: the time at which events actually occurred." / "Timestamps inserted into each record at the source." / "Processing time: the time when the record is received at the streaming application."

Question 3: What is the purpose of "windowing" in stream processing?

A. To guarantee that events arrive in perfect order.

B. To divide a continuous, unbounded stream into discrete, bounded chunks (windows) for computation.

C. To define when a computation within a window should be performed (triggering).

D. To assign a unique timestamp to each incoming data tuple.

Answer: B. To divide a continuous, unbounded stream into discrete, bounded chunks (windows) for computation.

Explanation: "Windowing: dividing a continuous stream of data into discrete chunks (windows)." This allows applying computations (like aggregations) over bounded sets of data within the unbounded stream.

Reference: Module 08 - Data Stream Processing - "Windowing: dividing a continuous stream of data into discrete chunks (windows)."

Question 4: What distinguishes a "tumbling window" from a "sliding window"?

A. Tumbling windows overlap, while sliding windows are disjoint.

B. Tumbling windows evict all tuples when the buffer fills (disjoint), while sliding windows evict older tuples as new ones arrive (overlapping).

C. Tumbling windows are based on event time, while sliding windows are based on processing time.

D. Tumbling windows use count-based triggers, while sliding windows use time-based triggers.

Answer: B. Tumbling windows evict all tuples when the buffer fills (disjoint), while sliding windows evict older tuples as new ones arrive (overlapping).

Explanation: The slides describe two possibilities: "tumbling and sliding". A "Tumbling window: when the buffer fills up, all the tuples are evicted." A "Sliding window: when the buffer fills up, older tuples are evicted." This implies tumbling windows are disjoint, while sliding windows have overlapping content.

Reference: Module 08 - Data Stream Processing - "Two possibilities: tumbling and sliding" / "Tumbling window: when the buffer fills up, all the tuples are evicted." / "Sliding window: when the buffer fills up, older tuples are evicted."

Question 5: What challenge arises specifically when processing data based on "event time"?

A. Event timestamps are always perfectly synchronized across all sources.

B. Processing based on event time eliminates the need for windowing.

C. Events can arrive out of order, making it difficult to determine when a window based on event time is complete.

D. Event time processing is typically much faster than processing time.

Answer: C. Events can arrive out of order, making it difficult to determine when a window based on event time is complete.

Explanation: Processing based on event time aims to "reflect the times at which events actually happened." However, this introduces the challenge of "Handling out-of-order events," as network latency or source variations can cause events to arrive at the processor later than events that occurred after them.

Reference: Module 08 - Data Stream Processing - "Reflect the times at which events actually happened." / "Handling out-of-order events."

Question 6: What is the role of "watermarks" in event time processing?

- A. Watermarks are used to define the boundaries of processing time windows.
- B. Watermarks flow with the stream and carry a timestamp t , declaring that event time has reached t and helping the system deal with lateness.
- C. Watermarks are special tuples that trigger computations immediately upon arrival.
- D. Watermarks are used to delete messages once they have been consumed.

Answer: B. Watermarks flow with the stream and carry a timestamp t , declaring that event time has reached t and helping the system deal with lateness.

Explanation: "Watermarking helps a stream processing system to deal with lateness." "Watermarks flow as part of the data stream and carry a timestamp t ." "A $W(t)$ declares that event time has reached time t in that stream," essentially acting as a "threshold to specify how long the system waits for late events."

Reference: Module 08 - Data Stream Processing - "Watermarking helps a stream processing system to deal with lateness." / "Watermarks flow as part of the data stream and carry a timestamp t ." / "A $W(t)$ declares that event time has reached time t in that stream" / "A watermark is a threshold to specify how long the system waits for late events."

Question 7: How does Spark Streaming (using DStreams) process a live data stream?

- A. It processes each event individually as it arrives using a continuous processing model.
- B. It runs the computation as a series of very small, deterministic batch jobs, treating each batch as an RDD.
- C. It treats the live stream as a continuously appended table (Structured Streaming).
- D. It forwards events directly to consumers without intermediate processing.

Answer: B. It runs the computation as a series of very small, deterministic batch jobs, treating each batch as an RDD.

Explanation: Spark Streaming uses a micro-batch approach. It aims to "Run a streaming computation as a series of very small, deterministic batch jobs." It "Chops up the live stream into batches of X seconds" and "Treats each batch as RDDs and processes them using RDD operations." This model is called "Discretized Stream Processing (DStream)".

Reference: Module 08 - Data Stream Processing - "Run a streaming computation as a series of very small, deterministic batch jobs." / "Chops up the live stream into batches of X seconds." / "Treats each batch as RDDs and processes them using RDD operations." / "Discretized Stream Processing (DStream)"

Question 8: What is a DStream in Spark Streaming?

- A. A continuously appended table representing the stream.
- B. A sequence of RDDs representing a stream of data.
- C. A connection to an external message broker like Kafka.
- D. A type of window used for event time processing.

Answer: B. A sequence of RDDs representing a stream of data.

Explanation: A "DStream: sequence of RDDs representing a stream of data." "Any operation applied on a DStream translates to operations on the underlying RDDs."

Reference: Module 08 - Data Stream Processing - "DStream: sequence of RDDs representing a stream of data." / "Any operation applied on a DStream translates to operations on the underlying RDDs."

Question 9: In Spark Streaming, when are transformations on DStreams actually computed?

- A. Immediately upon definition (eager evaluation).
- B. They are lazy and computation is kicked off explicitly by a call to the `start()` method on the `StreamingContext`.
- C. After each micro-batch interval automatically.
- D. Only when a watermark passes a certain threshold.

Answer: B. They are lazy and computation is kicked off explicitly by a call to the `start()` method on the `StreamingContext`.

Explanation: Just like RDD transformations, "Transformations on DStreams are still lazy!" The "Computation is kicked off explicitly by a call to the `start()` method." The `awaitTermination()`

method is then used to wait for the computation to finish.

Reference: Module 08 - Data Stream Processing - "Transformations on DStreams are still lazy!" / "Computation is kicked off explicitly by a call to the start() method." / "Start the computation and wait for it to terminate."

Question 10: Which DStream transformation aggregates values for each key within each RDD (micro-batch)?

- A. `map`
- B. `reduce`
- C. `reduceByKey`
- D. `reduceByWindow`

Answer: C. `reduceByKey`

Explanation: `reduceByKey` operates on a DStream of (K, V) pairs and returns "a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function" *within each RDD* of the DStream. `reduce` aggregates all elements in each RDD into a single element. `reduceByWindow` aggregates over multiple RDDs.

Reference: Module 08 - Data Stream Processing - "reduceByKey: a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function."

Question 11: How can you achieve a "tumbling window" effect using Spark Streaming's window transformations?

- A. By setting the `slideInterval` equal to the `windowLength`.
- B. By setting the `windowLength` to be much smaller than the `slideInterval`.
- C. By using `reduceByKey` instead of `reduceByKeyAndWindow`.
- D. By setting the micro-batch interval equal to the `windowLength`.

Answer: A. By setting the `slideInterval` equal to the `windowLength`.

Explanation: Window transformations take two parameters: `windowLength` and `slideInterval`. "A tumbling window effect can be achieved by making slide interval = window length".

Reference: Module 08 - Data Stream Processing - "A window is defined by two parameters: window length and slide interval." / "A tumbling window effect can be achieved by making slide interval = window

length"

Question 12: What is the core concept behind Spark's Structured Streaming?

- A. Processing streams as a sequence of micro-batch RDDs (DStreams).
- B. Treating a live data stream as a table that is being continuously appended.
- C. Using event time watermarks to handle out-of-order data in DStreams.
- D. Directly connecting producers and consumers without a message broker.

Answer: B. Treating a live data stream as a table that is being continuously appended.

Explanation: Structured Streaming uses a different model than DStreams. Its core concept is "Treating a live data stream as a table that is being continuously appended." This allows using DataFrame/Dataset APIs on streams.

Reference: Module 08 - Data Stream Processing - "Structured Streaming" / "Treating a live data stream as a table that is being continuously appended."

Question 13: In Structured Streaming, what does the "Append" output mode do?

- A. The entire updated result table is written to external storage on every trigger.
- B. Only the rows that were updated in the result table since the last trigger are changed in external storage.
- C. Only the *new* rows appended to the result table since the last trigger are written to external storage.
- D. It appends the raw input stream data directly to the output sink without processing.

Answer: C. Only the *new* rows appended to the result table since the last trigger are written to external storage.

Explanation: Structured Streaming has three output modes. "Append: only the new rows appended to the result table since the last trigger will be written to the external storage." This is suitable for queries where rows are only added, not updated (e.g., simple map/filter).

Reference: Module 08 - Data Stream Processing - "Three output modes:" / "1. Append: only the new rows appended to the result table since the last trigger will be written to the external storage."

Question 14: What is a primary disadvantage of "Direct Messaging" systems for stream dissemination?

- A. They introduce high latency.

- B. They require a central broker, creating a bottleneck.
- C. They are not durable; if a consumer crashes or goes offline, it misses messages.
- D. They cannot handle high message volumes.

Answer: C. They are not durable; if a consumer crashes or goes offline, it misses messages.

Explanation: Direct messaging requires producers and consumers "to be online at the same time." A key problem is durability: "What happens if a consumer crashes or temporarily goes offline? (not durable)". This leads to the need for message brokers.

Reference: Module 08 - Data Stream Processing - "Direct messaging" / "Both consumers and producers have to be online at the same time." / "What happens if a consumer crashes or temporarily goes offline? (not durable)"

Question 15: How does a "Message Broker" decouple producers and consumers?

- A. Producers send messages directly to consumers, but consumers acknowledge receipt via the broker.
- B. The broker runs as a server; producers write messages to it, and consumers read messages from it asynchronously.
- C. The broker assigns each producer to a specific consumer.
- D. The broker transforms messages from the producer's format to the consumer's format.

Answer: B. The broker runs as a server; producers write messages to it, and consumers read messages from it asynchronously.

Explanation: "A message broker decouples the producer-consumer interaction." "It runs as a server, with producers and consumers connecting to it as clients." "Producers write messages to the broker, and consumers receive them by reading them from the broker." "Consumers are generally asynchronous."

Reference: Module 08 - Data Stream Processing - "A message broker decouples the producer-consumer interaction." / "It runs as a server, with producers and consumers connecting to it as clients." / "Producers write messages to the broker, and consumers receive them by reading them from the broker." / "Consumers are generally asynchronous."

Question 16: What distinguishes a "Log-based message broker" (like Kafka) from typical message brokers?

- A. Log-based brokers delete messages immediately after consumption; typical brokers retain them.

B. Log-based brokers durably store all events in a sequential, append-only log, allowing multiple consumers or replays.

C. Log-based brokers only support direct messaging.

D. Log-based brokers are slower but more consistent.

Answer: B. Log-based brokers durably store all events in a sequential, append-only log, allowing multiple consumers or replays.

Explanation: "In typical message brokers, once a message is consumed, it is deleted." In contrast, "Log-based message brokers durably store all events in a sequential log." This log is an "append-only sequence of records on disk," enabling replayability and multiple independent consumers.

Reference: Module 08 - Data Stream Processing - "In typical message brokers, once a message is consumed, it is deleted." / "Log-based message brokers durably store all events in a sequential log." / "A log is an append-only sequence of records on disk."

Question 17: In Kafka, what is a "Topic"?

A. A single, globally ordered log file storing all messages.

B. A logical category or feed name ("queue") to which messages are published, typically split into partitions.

C. A consumer application that reads messages.

D. A Kafka server node (broker).

Answer: B. A logical category or feed name ("queue") to which messages are published, typically split into partitions.

Explanation: "Topics are queues: a stream of messages of a particular type". "Topics are logical collections of partitions (the physical files)." Producers publish to topics, and consumers subscribe to them.

Reference: Module 08 - Data Stream Processing - "Topics are queues: a stream of messages of a particular type" / "Topics are logical collections of partitions (the physical files)."

Question 18: What ordering guarantee does Kafka provide for messages within a topic?

A. Global ordering across all partitions of the topic.

B. Ordering is only guaranteed *within a single partition*.

C. Messages are ordered based on consumer processing time.

D. Kafka provides no ordering guarantees.

Answer: B. Ordering is only guaranteed *within a single partition*.

Explanation: "Ordering is only guaranteed within a partition for a topic." "Messages sent by a producer to a particular topic partition will be appended in the order they are sent." A consumer reading that partition "sees messages in the order they are stored in the log" for that partition only.

Reference: Module 08 - Data Stream Processing - "Ordering is only guaranteed within a partition for a topic." / "Messages sent by a producer to a particular topic partition will be appended in the order they are sent." / "A consumer instance sees messages in the order they are stored in the log."

Question 19: In Kafka's architecture, what is the role of the "leader" for a partition?

A. The leader is the consumer currently reading from that partition.

B. The leader is responsible for assigning partitions to brokers.

C. The leader is the designated broker replica for a partition; all writes and reads for that partition must go to the leader.

D. The leader maintains the watermark for event time processing.

Answer: C. The leader is the designated broker replica for a partition; all writes and reads for that partition must go to the leader.

Explanation: Partitions are replicated for fault tolerance across brokers. For each partition, "One broker is the leader". The rule is: "all writes and reads must go to the leader." Other replicas act as followers.

Reference: Module 08 - Data Stream Processing - "Partitions of a topic are replicated: fault-tolerance" / "One broker is the leader of a partition: all writes and reads must go to the leader."

Question 20: Which Spark Streaming function is used to create a DStream from data received over a TCP socket connection?

A. `textFileStream(directory)`

B. `kafkaStream(...)`

C. `socketTextStream(hostname, port)`

D. `readStream.format("socket") ...`

Answer: C. `socketTextStream(hostname, port)`

Explanation: The slides show an example using `ssc.socketTextStream("localhost", 9999)`. This "Creates a DStream from text data received over a TCP socket connection," specified by hostname and port. `readStream` is used for Structured Streaming.

Reference: Module 08 - Data Stream Processing - "Socket connection" / "Creates a DStream from text data received over a TCP socket connection." / (Code example showing `ssc.socketTextStream("localhost", 9999)`)

Module 10 – Large Scale Graph Processing (pp. 852–943)

Question 1: Why is parallelizing graph processing considered difficult compared to typical data-parallel tasks?

- A. Graphs are typically too small to benefit from parallelization.
- B. Graph algorithms are inherently sequential and cannot be parallelized.
- C. It's difficult to extract parallelism based on partitioning the data (graph partition is challenging), and computation often depends on neighbors across partitions.
- D. Graph processing frameworks do not support standard programming languages.

Answer: C. It's difficult to extract parallelism based on partitioning the data (graph partition is challenging), and computation often depends on neighbors across partitions.

Explanation: The slides highlight the difficulties: "Difficult to extract parallelism based on partitioning of the data," "Difficult to express parallelism based on partitioning of computation," and "Graph partition is a challenging problem." The interconnected nature of graphs makes simple data partitioning less effective.

Reference: Module 10 - Large Scale Graph Processing - "Difficult to extract parallelism based on partitioning of the data." / "Difficult to express parallelism based on partitioning of computation." / "Graph partition is a challenging problem."

Question 2: What are the two main goals of graph partitioning algorithms like vertex-cut and edge-cut?

- A. 1. Create partitions of exactly equal size; 2. Maximize the number of connections between partitions.
- B. 1. Create partitions of nearly equal size (load balancing); 2. Minimize communication (cut edges or replicated vertices).

C. 1. Group vertices with high degrees together; 2. Isolate vertices with low degrees.

D. 1. Ensure each partition fits in memory; 2. Randomly assign vertices/edges to partitions.

Answer: B. 1. Create partitions of nearly equal size (load balancing); 2. Minimize communication (cut edges or replicated vertices).

Explanation: The goal of partitioning is to "distribut[e] to hosts." Vertex-cut aims for "Nearly equal size (w.r.t. the number of vertices)" and "minimum number of edges that span separated clusters" (edge cuts). Edge-cut aims for "Nearly equal size (w.r.t. the number of edges)" and "minimum number of replicated vertices." Both balance size and minimize inter-partition dependencies.

Reference: Module 10 - Large Scale Graph Processing - "Vertex-cut partitioning" / "Nearly equal size (w.r.t. the number of vertices)." / "With the minimum number of edges that span separated clusters." / "Edge-cut partitioning" / "Nearly equal size (w.r.t. the number of edges)." / "With the minimum number of replicated vertices."

Question 3: Why do edge-cut partitioning algorithms often perform poorly on natural graphs with skewed Power-Law degree distributions?

A. Because power-law graphs have very few edges.

B. Because edge-cut algorithms require vertices to be evenly distributed, which power-law graphs lack.

C. Because high-degree vertices (hubs) in power-law graphs cause edge cuts to replicate these vertices many times, leading to high communication/storage overhead.

D. Because power-law graphs cannot be partitioned.

Answer: C. Because high-degree vertices (hubs) in power-law graphs cause edge cuts to replicate these vertices many times, leading to high communication/storage overhead.

Explanation: "Natural graphs: skewed Power-Law degree distribution." "Edge-cut algorithms perform poorly on Power-Law Graphs." While not explicitly stated why in this section, edge-cut partitioning minimizes replicated vertices. In power-law graphs, high-degree vertices have many edges; assigning these edges to different partitions (edge-cut) necessitates replicating the high-degree vertex across those partitions, which becomes inefficient. PowerGraph (mentioned later) addresses this.

Reference: Module 10 - Large Scale Graph Processing - "Natural graphs: skewed Power-Law degree distribution." / "Edge-cut algorithms perform poorly on Power-Law Graphs." (Conceptual link to PowerGraph/GAS model)

Question 4: What is a major drawback of implementing iterative algorithms like PageRank directly using MapReduce?

- A. MapReduce does not support key-value pairs.
- B. The invariant graph topology data needs to be re-loaded and re-processed at each iteration, wasting I/O, network bandwidth, and CPU.
- C. MapReduce cannot handle the shuffling required for PageRank updates.
- D. MapReduce jobs cannot run for more than one iteration.

Answer: B. The invariant graph topology data needs to be re-loaded and re-processed at each iteration, wasting I/O, network bandwidth, and CPU.

Explanation: "MapReduce does not directly support iterative algorithms." A key issue is that "Invariant graph-topology-data re-loaded and re-processed at each iteration is wasting I/O, network bandwidth, and CPU." Additionally, "Materializations of intermediate results at every MapReduce iteration harm performance."

Reference: Module 10 - Large Scale Graph Processing - "MapReduce does not directly support iterative algorithms." / "Invariant graph-topology-data re-loaded and re-processed at each iteration is wasting I/O, network bandwidth, and CPU" / "Materializations of intermediate results at every MapReduce iteration harm performance."

Question 5: What is the core idea behind the "Think Like a Vertex" (or graph-parallel) processing model?

- A. Each edge computes its value based on its connected vertices.
- B. The entire graph structure is processed sequentially by a single master node.
- C. Each vertex computes its value individually and in parallel, typically based on information from its neighbors.
- D. The graph is transformed into a table, and computations are done using SQL-like queries.

Answer: C. Each vertex computes its value individually and in parallel, typically based on information from its neighbors.

Explanation: The "Think Like a Vertex" model means "Each vertex computes individually its value (in parallel)." This "Computation typically depends on the neighbors." It is also known as the "graph-parallel processing model."

Reference: Module 10 - Large Scale Graph Processing - "Think Like a Vertex" / "Each vertex computes individually its value (in parallel)." / "Computation typically depends on the neighbors." / "Also know as graph-parallel processing model."

Question 6: In Google's Pregel system, computation proceeds in a sequence of iterations called "supersteps." What can a vertex do during a superstep S?

- A. Read messages sent in superstep S, send messages for superstep S+1, and modify its state.
- B. Read messages sent in superstep S-1, send messages for superstep S+1, and modify its state.
- C. Only read messages sent in superstep S-1.
- D. Only send messages to be received in superstep S+1.

Answer: B. Read messages sent in superstep S-1, send messages for superstep S+1, and modify its state.

Explanation: Pregel is inspired by the Bulk Synchronous Parallel (BSP) model. "Applications run in sequence of iterations, called supersteps." In superstep S, a vertex can: "reads messages sent to it in superstep S-1," "sends messages to other vertices: receiving at superstep S+1," and "modifies its state."

Reference: Module 10 - Large Scale Graph Processing - "Inspired by bulk synchronous parallel (BSP) model." / "Applications run in sequence of iterations, called supersteps." / "reads messages sent to it in superstep S-1." / "sends messages to other vertices: receiving at superstep S+1." / "modifies its state."

Question 7: When does a Pregel computation terminate?

- A. After a fixed number of supersteps.
- B. When all vertices are simultaneously inactive (voted to halt) and there are no messages in transit.
- C. When the master node decides to stop the computation.
- D. When any single vertex votes to halt.

Answer: B. When all vertices are simultaneously inactive (voted to halt) and there are no messages in transit.

Explanation: Vertices start active and can deactivate by "voting to halt." A halted vertex can become active again if it "receives a message." The algorithm terminates only when "All vertices are simultaneously inactive" AND "There are no messages in transit."

Reference: Module 10 - Large Scale Graph Processing - "A vertex deactivates itself by voting to halt" / "A halted vertex can be active if it receives a message." / "The whole algorithm terminates when:" / "All vertices are simultaneously inactive." / "There are no messages in transit."

Question 8: How does GraphLab differ from Pregel in its execution model?

- A. GraphLab uses a synchronous BSP model, while Pregel is asynchronous.
- B. GraphLab allows asynchronous iterative computation, while Pregel uses synchronous supersteps.
- C. GraphLab uses message passing, while Pregel uses shared memory.

D. GraphLab requires vertices to vote to halt, while Pregel does not.

Answer: B. GraphLab allows asynchronous iterative computation, while Pregel uses synchronous supersteps.

Explanation: The key difference highlighted is execution timing. "GraphLab allows asynchronous iterative computation," whereas Pregel uses the synchronous BSP model with supersteps. GraphLab also uses a shared memory approach within a vertex's scope, contrasting with Pregel's message passing.

Reference: Module 10 - Large Scale Graph Processing - "GraphLab allows asynchronous iterative computation." / Pregel: "Inspired by bulk synchronous parallel (BSP) model." / GraphLab: "(shared memory)" / Pregel: "Vertices communicate directly with one another by sending messages."

Question 9: What is the "vertex scope" in GraphLab?

- A. The messages received by a vertex in the previous superstep.
- B. The data stored only in the vertex itself.
- C. The data stored in a vertex, and in all adjacent vertices and edges.
- D. The partition to which a vertex belongs.

Answer: C. The data stored in a vertex, and in all adjacent vertices and edges.

Explanation: The "Vertex scope of vertex v: the data stored in v, and in all adjacent vertices and edges." A vertex program can "read and modify any of the data in its scope (shared memory)."

Reference: Module 10 - Large Scale Graph Processing - "Vertex scope of vertex v: the data stored in v, and in all adjacent vertices and edges." / "A vertex can read and modify any of the data in its scope (shared memory)."

Question 10: What are the three phases of the Gather-Apply-Scatter (GAS) model, often associated with GraphLab/PowerGraph?

- A. Map, Shuffle, Reduce
- B. Read, Write, Commit
- C. Gather (accumulate from neighbors), Apply (update self), Scatter (update neighbors)
- D. VoteToHalt, SendMessages, ModifyState

Answer: C. Gather (accumulate from neighbors), Apply (update self), Scatter (update neighbors)

Explanation: The GAS model "Factorizes the local vertices functions into the Gather, Apply and Scatter phases." "Gather: accumulate information from neighborhood." "Apply: apply the accumulated value to center vertex." "Scatter: update adjacent edges and vertices."

Reference: Module 10 – Large Scale Graph Processing – "Factorizes the local vertices functions into the Gather, Apply and Scatter phases." / "Gather: accumulate information from neighborhood." / "Apply: apply the accumulated value to center vertex." / "Scatter: update adjacent edges and vertices."

Question 11: What limitation of pure graph-parallel systems led to the development of systems like GraphX?

- A. Graph-parallel systems cannot perform iterative computations.
- B. Graph-parallel systems are too slow for large graphs.
- C. The restrictions that enable performance make it difficult/inefficient to express other stages in a typical graph-analytics pipeline (e.g., data loading, transformation).
- D. Graph-parallel systems can only process graphs that fit in a single machine's memory.

Answer: C. The restrictions that enable performance make it difficult/inefficient to express other stages in a typical graph-analytics pipeline (e.g., data loading, transformation).

Explanation: Graph-parallel computation involves "restricting the types of computation to achieve performance." However, "The same restrictions make it difficult and inefficient to express many stages in a typical graph-analytics pipeline." GraphX aims to address this by unifying data-parallel and graph-parallel views.

Reference: Module 10 – Large Scale Graph Processing – "Graph-parallel computation: restricting the types of computation to achieve performance." / "The same restrictions make it difficult and inefficient to express many stages in a typical graph-analytics pipeline."

Question 12: How does Spark GraphX represent a graph?

- A. As a single adjacency matrix RDD.
- B. As a pair of collections: a VertexRDD containing vertex properties and an EdgeRDD containing edge properties.
- C. As a Pregel-like vertex program.
- D. As a relational table with source, destination, and property columns.

Answer: B. As a pair of collections: a VertexRDD containing vertex properties and an EdgeRDD containing edge properties.

Explanation: Spark represents graph data as a "property graph." Logically, "It is logically represented as a pair of vertex and edge property collections." These are the "VertexRDD: contains the vertex properties..." and the "EdgeRDD: contains the edge properties..."

Reference: Module 10 – Large Scale Graph Processing – "Spark represent graph structured data as a property graph." / "It is logically represented as a pair of vertex and edge property collections." / "VertexRDD: contains the vertex properties..." / "EdgeRDD: contains the edge properties..."

Question 13: What is the "triplets" collection in GraphX?

- A. An RDD containing only vertices that have exactly three neighbors.
- B. An RDD that logically joins each edge with its corresponding source and destination vertex properties (`RDD[EdgeTriplet[VD, ED]]`).
- C. The intermediate messages passed between vertices during Pregel computation.
- D. A collection storing the Gather, Apply, and Scatter functions.

Answer: B. An RDD that logically joins each edge with its corresponding source and destination vertex properties (`RDD[EdgeTriplet[VD, ED]]`).

Explanation: "The triplets collection consists of each edge and its corresponding source and destination vertex properties." "It logically joins the vertex and edge properties: `RDD[EdgeTriplet[VD, ED]]`." The `EdgeTriplet` class adds `srcAttr` and `dstAttr` to the basic `Edge` class.

Reference: Module 10 – Large Scale Graph Processing – "The triplets collection consists of each edge and its corresponding source and destination vertex properties." / "It logically joins the vertex and edge properties: `RDD[EdgeTriplet[VD, ED]]`." / "The `EdgeTriplet` class extends the `Edge` class by adding the `srcAttr` and `dstAttr` members..."

Question 14: What critique is raised regarding PageRank's definition of "importance"?

- A. It is too computationally expensive to calculate.
- B. It treats popularity (link counts) as importance, ignoring meaning, quality, intent, and social context.
- C. It gives too much weight to recently created pages.
- D. It cannot be implemented using the "Think Like a Vertex" model.

Answer: B. It treats popularity (link counts) as importance, ignoring meaning, quality, intent, and social context.

Explanation: The slides critique the claim that numbers "speak for themselves." PageRank "ranks nodes by link counts" and "treats popularity (links) as importance: ignores meaning, quality, or intent behind

connections." This risks making "Marginalized or less-connected voices... invisibility" and erases "Context (history, culture, power)".

Reference: Module 10 – Large Scale Graph Processing – "PageRank ranks nodes by link counts in this spirit." / "PageRank treats popularity (links) as importance: ignores meaning, quality, or intent behind connections." / "Marginalized or less-connected voices risk invisibility..." / "Context (history, culture, power) is erased in favor of raw link counts."

Question 15: When partitioning graphs for scalability, what social aspect might be negatively impacted?

- A. The total number of vertices might decrease.
- B. Algorithms optimize for balance and cut-size, potentially splitting peripheral groups or breaking cross-community ties (weak ties).
- C. The PageRank scores of all vertices might become uniform.
- D. The graph might become disconnected.

Answer: B. Algorithms optimize for balance and cut-size, potentially splitting peripheral groups or breaking cross-community ties (weak ties).

Explanation: The critique asks "what social ties might be lost?". The answer suggests "Algorithms typically optimize for balance and cut-size, not social meaning." This can lead to "Peripheral or minority groups may be split across partitions" and "Cross-community ties (weak ties) can be broken: erasing nuance."

Reference: Module 10 – Large Scale Graph Processing – "When we partition graphs for scalability, what social ties might be lost?" / "Algorithms typically optimize for balance and cut-size, not social meaning." / "Peripheral or minority groups may be split across partitions." / "Cross-community ties (weak ties) can be broken: erasing nuance."

Question 16: In the context of running PageRank on MapReduce, what does the Map function typically emit?

- A. For each page i , its current rank $R[i]$ divided by its number of outgoing links, emitted for each neighbor j .
- B. For each page i , a list of its neighbors $Nbrs(i)$.
- C. For each page j , the sum of ranks received from its neighbors.
- D. A single value representing the convergence status.

Answer: A. For each page i , its current rank $R[i]$ divided by its number of outgoing links, emitted for each neighbor j .

Explanation: The PageRank formula is $R[i] = \sum_{j \in \text{Nbrs}(i)} w_{ji} R[j]$. The Map function needs to distribute the current rank $R[j]$ of page j to its neighbors i . The weight w_{ji} is typically $1/|\text{Out}(j)|$. The visualization shows the map function taking page j and its rank $R[j]$ and emitting $(i, R[j] / |\text{Out}(j)|)$ for each neighbor i .

Reference: Module 10 – Large Scale Graph Processing – (PageRank formula) / (Map function diagram showing rank distribution to neighbors)

Question 17: What does the Reduce function typically do in a MapReduce implementation of PageRank?

- A. It calculates the number of outgoing links for each page.
- B. It distributes the current rank of a page to its neighbors.
- C. For each page i , it sums up the partial ranks received from all its neighbors j to compute the new rank $R[i]$.
- D. It checks if the ranks have converged.

Answer: C. For each page i , it sums up the partial ranks received from all its neighbors j to compute the new rank $R[i]$.

Explanation: After the shuffle phase groups all partial ranks sent to page i , the Reduce function receives $(i, [\text{partial_rank1}, \text{partial_rank2}, \dots])$. Its job is to sum these partial ranks according to the formula $R[i] = \sum_{j \in \text{Nbrs}(i)} w_{ji} R[j]$ to calculate the new rank for page i .

Reference: Module 10 – Large Scale Graph Processing – (PageRank formula) / (Reduce function diagram showing summation of incoming ranks)

Question 18: Which system aims to unify data-parallel and graph-parallel processing by treating tables and graphs as composable views of the same physical data?

- A. Pregel
- B. MapReduce
- C. GraphLab
- D. GraphX

Answer: D. GraphX

Explanation: GraphX is presented under the "Think Like a Table" section. It "Unifies data-parallel and graph-parallel systems." The core idea is that "Tables and Graphs are composable views of the same physical data."

Reference: Module 10 – Large Scale Graph Processing – "Think Like a Table" / "Unifies data-parallel and graph-parallel systems." / "Tables and Graphs are composable views of the same physical data."

Question 19: What system introduced the Gather-Apply-Scatter (GAS) model for vertex computations?

- A. Pregel
- B. GraphLab/PowerGraph
- C. GraphX
- D. MapReduce

Answer: B. GraphLab/PowerGraph

Explanation: The GAS model is described in the section on GraphLab/Turi. PowerGraph (developed by the GraphLab team) explicitly "Factorizes the local vertices functions into the Gather, Apply and Scatter phases."

Reference: Module 10 – Large Scale Graph Processing – "Factorizes the local vertices functions into the Gather, Apply and Scatter phases." (Associated with GraphLab/PowerGraph)

Question 20: If a vertex in Pregel votes to halt but then receives a message in a later superstep, what happens?

- A. The vertex remains halted, and the message is discarded.
- B. The entire computation terminates immediately.
- C. The vertex becomes active again for the next superstep.
- D. The sending vertex receives an error message.

Answer: C. The vertex becomes active again for the next superstep.

Explanation: "A vertex deactivates itself by voting to halt". However, "A halted vertex can be active if it receives a message." This ensures that updates can propagate through the graph even if parts of it appeared locally converged.

Reference: Module 10 – Large Scale Graph Processing – "A vertex deactivates itself by voting to halt: no further work to do." / "A halted vertex can be active if it receives a message."

Module 11 – Resource Management – Mesos, YARN, and Borg (pp. 944–1001)

Question 1: What is the primary motivation for running multiple distributed frameworks (like Hadoop, Spark, etc.) on a single shared cluster?

- A. To increase the complexity of cluster management.
- B. To maximize resource utilization and allow data sharing between frameworks, avoiding the cost of dedicated clusters.
- C. To ensure that each framework runs in complete isolation without sharing any resources.
- D. To test the fault tolerance capabilities of the underlying hardware.

Answer: B. To maximize resource utilization and allow data sharing between frameworks, avoiding the cost of dedicated clusters.

Explanation: Running each framework on a dedicated cluster is "Expensive" and makes it "Hard to share data." The goal of running multiple frameworks on one cluster is to "Maximize utilization and share data between frameworks."

Reference: Module 11 - Resource Management - "Running each framework on its dedicated cluster:" / "Expensive" / "Hard to share data" / "Running multiple frameworks on a single cluster." / "Maximize utilization and share data between frameworks."

Question 2: What is the key difference between a "Monolithic scheduler" and a "Two-Level scheduler"?

- A. Monolithic schedulers are simple and easy to scale; Two-Level schedulers are complex but optimal.
- B. Monolithic schedulers use a single, centralized algorithm for all jobs; Two-Level schedulers separate resource allocation (manager) from task placement (framework scheduler).
- C. Monolithic schedulers support only one framework; Two-Level schedulers support multiple frameworks.
- D. Monolithic schedulers are request-based; Two-Level schedulers are offer-based.

Answer: B. Monolithic schedulers use a single, centralized algorithm for all jobs; Two-Level schedulers separate resource allocation (manager) from task placement (framework scheduler).

Explanation: "Monolithic schedulers: use a single, centralized scheduling algorithm for all jobs." "Two-level schedulers: separate concerns of resource allocation and task placement." In the two-level model, a resource manager offers resources to framework schedulers, which then decide how to use them.

Reference: Module 11 - Resource Management - "Monolithic scheduler" / "Monolithic schedulers: use a single, centralized scheduling algorithm for all jobs." / "Two-Level scheduler" / "Two-level schedulers: separate concerns of resource allocation and task placement."

Question 3: Which systems are examples of Two-Level Schedulers?

- A. Borg
- B. Mesos and YARN
- C. Kubernetes
- D. Docker

Answer: B. Mesos and YARN

Explanation: The slides categorize the systems: "Two-level schedulers: ... Mesos and Yarn". "Monolithic schedulers: ... Borg".

Reference: Module 11 - Resource Management - "Two-level schedulers: ... Mesos and Yarn" / "Monolithic schedulers: ... Borg"

Question 4: In Apache Mesos, what is a "resource offer"?

- A. A request from a framework for specific resources.
- B. A vector of available resources (e.g., CPU, RAM) on a specific node, offered by the Master to a framework.
- C. The total amount of resources available in the entire cluster.
- D. A task submitted by a framework to run on a slave node.

Answer: B. A vector of available resources (e.g., CPU, RAM) on a specific node, offered by the Master to a framework.

Explanation: Mesos uses an offer-based model. The "Master sends resource offers to frameworks." The "Unit of allocation: resource offer" is a "Vector of available resources on a node", for example, `node1: <1CPU, 1GB>`.

Reference: Module 11 - Resource Management - "Master sends resource offers to frameworks." / "Unit of allocation: resource offer" / "Vector of available resources on a node"

Question 5: What is the core principle of "Max-Min Fairness"?

- A. Allocate resources proportionally based on user-defined weights only.

B. Give all resources to the user with the highest demand.

C. Maximize the minimum allocation received by any user, handling cases where users want less than their equal share.

D. Minimize the maximum allocation received by any user.

Answer: C. Maximize the minimum allocation received by any user, handling cases where users want less than their equal share.

Explanation: Max-min fairness generalizes the idea of equal sharing ("allocate each $1/n$ "). It aims to maximize the minimum share anyone gets, but also "Handles if a user wants less than its fair share."

Reference: Module 11 - Resource Management - "Generalized by max-min fairness." / "Handles if a user wants less than its fair share."

Question 6: Why is simple Max-Min Fairness sometimes insufficient when allocating multiple heterogeneous resource types (e.g., CPU and Memory)?

A. It doesn't allow users to specify different demands for different resources.

B. It can violate the "share guarantee" because a user might receive less than an equal fraction of *all* resource types.

C. It is not strategy-proof; users benefit from asking for more resources than they need.

D. It cannot handle weighted allocations based on user importance.

Answer: B. It can violate the "share guarantee" because a user might receive less than an equal fraction of *all* resource types.

Explanation: When dealing with multiple resources, simply equalizing one resource might lead to unfairness in others. "Asset fairness" attempts to equalize total value but "violates share grantee." A user might get less than 50% of both CPU and RAM, meaning they'd be "Better off in a separate cluster with half the resources."

Reference: Module 11 - Resource Management - "When is Max-Min Fairness NOT Enough?" / "Need to schedule multiple, heterogeneous resources..." / "Asset fairness yields..." / "Problem: violates share grantee." / "User 1 gets less than 50% of both CPU and RAM."

Question 7: What is the key idea behind Dominant Resource Fairness (DRF)?

A. Give priority to users whose tasks require the most CPU.

B. Allocate resources based only on the user's memory requirements.

C. Identify each user's "dominant resource" (the one they have the biggest share of) and apply max-min fairness to these "dominant shares".

D. Ignore dominant resources and allocate equal fractions of all resources to all users.

Answer: C. Identify each user's "dominant resource" (the one they have the biggest share of) and apply max-min fairness to these "dominant shares".

Explanation: DRF aims to generalize max-min fairness to multiple resources. It defines the "Dominant resource of a user: the resource that user has the biggest share of" and their "Dominant share". The policy is to "Apply max-min fairness to dominant shares: give every user an equal share of her dominant resource," effectively equalizing the dominant shares across users.

Reference: Module 11 - Resource Management - "Dominant Resource Fairness (DRF)" / "Dominant resource of a user: the resource that user has the biggest share of." / "Apply max-min fairness to dominant shares: give every user an equal share of her dominant resource." / "Equalize the dominant share of the users."

Question 8: In Apache YARN's architecture, what are the main components?

A. Master, Slave, Framework Scheduler, Executor

B. BorgMaster, Borglet, Alloc

C. Resource Manager (RM), Application Master (AM), Node Manager (NM)

D. Driver, Executor, SparkContext

Answer: C. Resource Manager (RM), Application Master (AM), Node Manager (NM)

Explanation: The main components listed for YARN are: "Resource Manager (RM)", "Application Master (AM)", and "Node Manager (NM)".

Reference: Module 11 - Resource Management - "YARN" (slide title) / "Resource Manager (RM)" / "Application Master (AM)" / "Node Manager (NM)"

Question 9: What is the role of the Application Master (AM) in YARN?

A. It manages the global resource allocation across the entire cluster.

B. It is the worker daemon running on each node, reporting resources to the RM.

C. It is the "head" of a specific job, runs as a container, and requests resources (containers) from the RM for its job.

D. It submits job requests from users directly to the Node Managers.

Answer: C. It is the "head" of a specific job, runs as a container, and requests resources (containers) from the RM for its job.

Explanation: The AM is "The head of a job." It "Runs as a container" and is responsible to "Request resources from RM (num. of containers/resource per container/locality ...)".

Reference: Module 11 - Resource Management - "Application Master (AM)" / "The head of a job." / "Runs as a container." / "Request resources from RM..."

Question 10: Which resource management system is described as the cluster management system used internally at Google?

- A. Mesos
- B. YARN
- C. Borg
- D. Kubernetes

Answer: C. Borg

Explanation: Borg is introduced as the "Cluster management system at Google." Kubernetes is later mentioned as being "based on Borg".

Reference: Module 11 - Resource Management - "Borg" (slide title) / "Cluster management system at Google." / "Kubernetes (based on Borg)"

Question 11: In Borg, what is a "cell"?

- A. The central controller that holds the cluster state.
- B. A reserved set of resources where a job can run.
- C. A set of machines managed by Borg as one unit.
- D. An agent running on each machine to manage tasks.

Answer: C. A set of machines managed by Borg as one unit.

Explanation: A "Cell: a set of machines managed by Borg as one unit."

Reference: Module 11 - Resource Management - "Cell: a set of machines managed by Borg as one unit."

Question 12: What is the difference between a Virtual Machine (VM) and a Container?

- A. VMs run multiple OS kernels, while Containers run a single OS kernel on the host.

B. Containers have relaxed isolation properties and share the host OS kernel, making them more lightweight than VMs which include their own OS.

C. VMs package applications with code and libraries, while Containers only package the OS.

D. Containers provide stronger security isolation than VMs.

Answer: B. Containers have relaxed isolation properties and share the host OS kernel, making them more lightweight than VMs which include their own OS.

Explanation: "Virtual Machines (VMs): a full machine running all the components, including its own operating system (OS)". "Containers are similar to VMs, but they have relaxed isolation properties to share the OS among the applications." This shared OS makes containers lighter.

Reference: Module 11 - Resource Management - "Virtual Machines (VMs): a full machine running all the components, including its own operating system (OS)..." / "Containers are similar to VMs, but they have relaxed isolation properties to share the OS among the applications."

Question 13: What is the role of Container Orchestration tools like Kubernetes?

A. To build container images from application code.

B. To provide a runtime environment for a single container.

C. To manage the lifecycles of containers (deployment, scaling, scheduling) in large, dynamic environments.

D. To virtualize hardware for running VMs.

Answer: C. To manage the lifecycles of containers (deployment, scaling, scheduling) in large, dynamic environments.

Explanation: Managing many containers manually is challenging ("what if we have 1000 containers and 400 services?"). "Container orchestration can help to manage the lifecycles of containers, especially in large and dynamic environments." Kubernetes and Marathon are listed as examples.

Reference: Module 11 - Resource Management - "Container scalability is an operational challenge." / "Container orchestration can help to manage the lifecycles of containers, especially in large and dynamic environments." / "Container orchestration tools: Kubernetes..., Marathon..."

Question 14: In Borg's architecture, what is the "Borglet"?

A. The central controller responsible for scheduling tasks.

B. A user-submitted job consisting of multiple tasks.

C. An agent running on each machine that manages and monitors tasks and resources, reporting to the BorgMaster.

D. A reserved set of resources on a machine.

Answer: C. An agent running on each machine that manages and monitors tasks and resources, reporting to the BorgMaster.

Explanation: The Borglet's role is to "Manage and monitor tasks and resource". The "BorgMaster polls Borglet every few seconds" to get status updates.

Reference: Module 11 - Resource Management - "Borglet" / "Manage and monitor tasks and resource" / "BorgMaster polls Borglet every few seconds"

Question 15: What potential bias is discussed regarding the setting of weights in "Weighted Max-Min Fairness"?

A. Weights are always set fairly based on resource needs.

B. System admins or organizational priorities might favor powerful users or well-funded areas, reinforcing existing advantages.

C. The weights can only be integers, limiting fairness options.

D. Users always lie about their needs to get higher weights.

Answer: B. System admins or organizational priorities might favor powerful users or well-funded areas, reinforcing existing advantages.

Explanation: The question asks "Who gets to decide weights... and what biases might this introduce?". Potential biases include: "System admins: may favor powerful users or paying customers", "Organizational priorities: research areas with more funding may get higher weights", leading to a "Bias risk: dominant groups reinforce their advantage".

Reference: Module 11 - Resource Management - "Who gets to decide weights in weighted max-min fairness and what biases might this introduce?" / "System admins: may favor powerful users..." / "Organizational priorities: research areas with more funding..." / "Bias risk: dominant groups reinforce their advantage..."

Question 16: How does Mesos's scheduling process work?

A. Frameworks request specific resources, and the Master grants them.

B. The Master makes resource offers to frameworks; frameworks select offers and tell the Master which tasks to run on those resources.

C. All frameworks submit tasks to a central queue, and the Master assigns them based on priority.

D. Slaves directly offer their resources to frameworks without involving the Master.

Answer: B. The Master makes resource offers to frameworks; frameworks select offers and tell the Master which tasks to run on those resources.

Explanation: Mesos uses a two-level, offer-based approach. "Master sends resource offers to frameworks." Then, "Frameworks select which offers to accept and which tasks to run." The "Framework scheduler selects resources and provides tasks."

Reference: Module 11 - Resource Management - "Master sends resource offers to frameworks." / "Frameworks select which offers to accept and which tasks to run." / "Framework scheduler selects resources and provides tasks."

Question 17: What is the primary function of the Node Manager (NM) in YARN?

A. To accept job submissions from users.

B. To coordinate resource requests between Application Masters.

C. To manage containers on a specific node, monitor their resource usage, and report status to the Resource Manager.

D. To schedule tasks within a container.

Answer: C. To manage containers on a specific node, monitor their resource usage, and report status to the Resource Manager.

Explanation: The NM is the "worker daemon" running "One per node." It "Registers with RM" and "Report[s] resources to RM". It is responsible for launching and managing containers on its node as directed by the AM (via the RM).

Reference: Module 11 - Resource Management - "Node Manager (NM)" / "The worker daemon." / "One per node." / "Registers with RM." / "Report resources to RM..."

Question 18: Which property of a fairness policy ensures that users are not better off by asking for more resources than they actually need?

A. Share guarantee

B. Strategy-proofness

C. Asset fairness

D. Dominant Resource Fairness

Answer: B. Strategy-proofness

Explanation: "Strategy proof" means "Users are not better off by asking for more than they need." Therefore, "Users have no reason to lie" about their requirements. Both Max-Min Fairness and DRF aim to be strategy-proof.

Reference: Module 11 - Resource Management - "Strategy proof" / "Users are not better off by asking for more than they need." / "Users have no reason to lie."

Question 19: According to the slides, how could resource management systems be designed to account for social context, such as prioritizing under-resourced groups?

- A. By strictly enforcing equal shares for all users using Max-Min Fairness.
- B. By allocating extra resources to groups with fewer starting advantages or scheduling based on social impact rather than just efficiency.
- C. By using monolithic schedulers, which can achieve optimal technical schedules.
- D. By implementing Docker containers for better resource isolation.

Answer: B. By allocating extra resources to groups with fewer starting advantages or scheduling based on social impact rather than just efficiency.

Explanation: The slides ask how to design systems for social context. Suggestions include: "Allocate extra resources to groups with fewer starting advantages" and "Scheduling that considers deadlines, social impact, or community benefit, not just efficiency."

Reference: Module 11 - Resource Management - "How could we design systems that account for social context...?" / "Allocate extra resources to groups with fewer starting advantages." / "Scheduling that considers deadlines, social impact, or community benefit, not just efficiency."

Question 20: What fundamental operational challenge arises when managing a large number of containers manually?

- A. Building the initial container images.
- B. Ensuring containers share the host OS kernel efficiently.
- C. Managing the deployment, scaling, maintenance, and lifecycle of numerous containers in dynamic environments.
- D. Migrating applications from VMs to containers.

Answer: C. Managing the deployment, scaling, maintenance, and lifecycle of numerous containers in dynamic environments.

Explanation: While managing a few containers is easy, "what if we have 1000 containers and 400 services?". This "Container scalability is an operational challenge." This challenge is what "Container orchestration" tools like Kubernetes address by helping "manage the lifecycles of containers".

Reference: Module 11 - Resource Management - "Container scalability is an operational challenge." / "But, what if we have 1000 containers and 400 services?" / "Container orchestration can help to manage the lifecycles of containers..."

Module 12 – Cloud Data Lakes (pp. 1002–1107)

Question 1: What were the primary limitations of traditional Data Warehouses that led to the rise of Data Lakes?

- A. They could not execute SQL queries efficiently.
- B. They were too expensive for small datasets and supported only unstructured data.
- C. They struggled with rapidly growing unstructured/semi-structured data, had high storage costs for large datasets, and lacked support for data science/ML.
- D. They did not support ACID transactions or time travel features.

Answer: C. They struggled with rapidly growing unstructured/semi-structured data, had high storage costs for large datasets, and lacked support for data science/ML.

Explanation: Data Warehouses "Could not support rapidly growing unstructured and semi-structured data," had "High cost to store large datasets," and offered "No support for data science and ML."

Reference: Module 12 - Cloud Data Lakes - "Could not support rapidly growing unstructured and semi-structured data..." / "High cost to store large datasets." / "No support for data science and ML."

Question 2: What is the core concept of a Data Lake?

- A. A purpose-built system optimized only for SQL analytics and BI.
- B. A low-cost storage system (like S3 or HDFS) designed to hold all raw data, regardless of structure, often using schema-on-read.
- C. A metadata layer that adds ACID transactions to existing data warehouses.
- D. A system that performs ETL directly from operational databases into structured tables.

Answer: B. A low-cost storage system (like S3 or HDFS) designed to hold all raw data, regardless of structure, often using schema-on-read.

Explanation: A Data Lake uses "Low-cost storage to hold all raw data, e.g., Amazon S3, and HDFS." It "stores all data irrespective of the source and its structure" and typically "defines the schema after data is stored" (schema-on-read).

Reference: Module 12 - Cloud Data Lakes - "Low-cost storage to hold all raw data, e.g., Amazon S3, and HDFS." / "Data Lake stores all data irrespective of the source and its structure" / "Data Lake defines the schema after data is stored"

Question 3: What is a major challenge associated with the two-tiered Data Lake + Data Warehouse architecture?

- A. Data Lakes cannot store unstructured data.
- B. Data Warehouses are cheaper than Data Lakes for large volumes.
- C. Increased complexity, potential data reliability issues due to extra ETL steps, and data staleness in the warehouse.
- D. Lack of support for SQL queries in both systems.

Answer: C. Increased complexity, potential data reliability issues due to extra ETL steps, and data staleness in the warehouse.

Explanation: While cheap storage is an advantage, the combined architecture is "much more complex." "Data reliability suffers" due to "Multiple storage systems" and "Extra ETL steps that can go wrong." "Timeliness suffers" because of these extra ETL steps.

Reference: Module 12 - Cloud Data Lakes - "system architecture is much more complex!" / "Data reliability suffers:" / "Multiple storage systems..." / "Extra ETL steps that can go wrong." / "Timeliness suffers and high cost:" / "Extra ETL steps before data is available..."

Question 4: How does a "Lakehouse" architecture attempt to combine the benefits of Data Lakes and Data Warehouses?

- A. By implementing Data Warehouse management features (like ACID, versioning) and performance directly on top of low-cost data lake storage using open formats.
- B. By replicating all data from the Data Lake into a highly optimized Data Warehouse in real-time.
- C. By replacing SQL with machine learning interfaces for all data access.
- D. By enforcing a strict schema-on-write for all data entering the Data Lake.

Answer: A. By implementing Data Warehouse management features (like ACID, versioning) and performance directly on top of low-cost data lake storage using open formats.

Explanation: "Lakehouse systems combine the benefits of Data Warehouses and Data Lakes". They aim to "Implement Data Warehouse management and performance features on top of directly-accessible data in open formats" stored in the data lake.

Reference: Module 12 - Cloud Data Lakes - "Lakehouse systems combine the benefits of Data Warehouses and Data Lakes..." / "Implement Data Warehouse management and performance features on top of directly-accessible data in open formats."

Question 5: Which technology is presented as an open-source storage layer that brings reliability (ACID transactions, versioning) to Data Lakes, forming a key component of a Lakehouse?

- A. Apache Spark
- B. Apache Parquet
- C. Delta Lake
- D. Amazon S3

Answer: C. Delta Lake

Explanation: "Delta Lake is an open source storage layer that brings reliability to Data Lakes." It "Provides ACID transactions," "Provides time travel and versioning," and "Unifies streaming and batch data processing." Metadata layers like Delta Lake are key to the Lakehouse concept.

Reference: Module 12 - Cloud Data Lakes - "Delta Lake is an open source storage layer that brings reliability to Data Lakes." / "Provides ACID transactions." / "Provides time travel and versioning." / "Unifies streaming and batch data processing." / "Metadata layers for Data Lakes" / "Implemented in multiple systems, such as Delta Lake."

Question 6: What is the DeltaLog in Delta Lake?

- A. A cache for hot data stored in RAM/SSD.
- B. A transaction log that records all changes (commits) made to a Delta Lake table as ordered, atomic JSON files.
- C. The schema definition stored in JSON format.
- D. An auxiliary data structure like statistics or indexes for query optimization.

Answer: B. A transaction log that records all changes (commits) made to a Delta Lake table as ordered, atomic JSON files.

Explanation: The "DeltaLog is a transaction log that tracks all changes that users make to the table." "Any changes... are then recorded as ordered, atomic commits in the DeltaLog." "Each commit is written

out as a JSON file". Delta Lake uses this log for ACID, time travel, etc.

Reference: Module 12 - Cloud Data Lakes - "DeltaLog is a transaction log that tracks all changes that users make to the table." / "Any changes to that table are then recorded as ordered, atomic commits in the DeltaLog." / "Each commit is written out as a JSON file..."

Question 7: How does the DeltaLog enable "time travel" (querying previous versions of a table)?

- A. By storing complete physical snapshots of the table after every commit.
- B. By allowing queries to specify a maximum latency, returning older data if necessary.
- C. Because the DeltaLog records every commit as a sequence of actions (add/remove file), the state of the table at any previous commit version can be reconstructed by processing the log up to that point.
- D. Time travel is only possible by restoring backups from S3.

Answer: C. Because the DeltaLog records every commit as a sequence of actions (add/remove file), the state of the table at any previous commit version can be reconstructed by processing the log up to that point.

Explanation: "Every table is the result of the sum of all of the commits recorded in the... DeltaLog." It "provides a step-by-step instruction guide... detailing exactly how to get from the table's original state to its current state." "Thus, we can recreate the state of a table at any point in time" by "processing only commits made prior to that point." This ability is "known as time travel or data versioning."

Reference: Module 12 - Cloud Data Lakes - "Every table is the result of the sum of all of the commits..." / "The DeltaLog provides a step-by-step instruction guide..." / "Thus, we can recreate the state of a table at any point in time." / "Starting with an original table, and processing only commits made prior to that point." / "This ability is known as time travel or data versioning."

Question 8: What is "Schema Enforcement" in Delta Lake?

- A. The ability to automatically add new columns to the table schema when writing data (Schema Evolution).
- B. The process of saving the table's schema in JSON format inside the DeltaLog.
- C. A feature that prevents writes if the data being written has a schema incompatible (e.g., wrong column types, extra columns) with the target table's schema.
- D. A governance tool for auditing schema changes over time.

Answer: C. A feature that prevents writes if the data being written has a schema incompatible (e.g., wrong column types, extra columns) with the target table's schema.

Explanation: "Schema enforcement: prevents users from accidentally polluting their tables with mistakes or garbage data." It "occurs on write." "If the schema is not compatible, Delta Lake cancels the transaction... no data is written," and "raises an exception". Compatibility rules include no extra columns and matching data types.

Reference: Module 12 - Cloud Data Lakes - "Schema enforcement: prevents users from accidentally polluting their tables with mistakes or garbage data." / "Schema enforcement (a.k.a schema validation) occurs on write." / "If the schema is not compatible, Delta Lake cancels the transaction..." / "Rule 1: cannot contain any additional columns..." / "Rule 2: cannot have column data types that differ..."

Question 9: How can you enable "Schema Evolution" in Delta Lake to allow adding new columns during an append operation?

- A. It is enabled by default; no action is needed.
- B. By setting the write option `mergeSchema` to `true`.
- C. By manually altering the table schema before the write operation.
- D. By using the `UPDATE` command instead of appending.

Answer: B. By setting the write option `mergeSchema` to `true`.

Explanation: Schema enforcement normally prevents adding new columns. "Schema evolution: enables automatic addition of columns when desired." A write with an extra column will normally fail, but "A new column can be explicitly added by setting the option `mergeSchema` to `true`."

Reference: Module 12 - Cloud Data Lakes - "Schema evolution: enables automatic addition of columns when desired." / "Rule 1: cannot contain any additional columns..." / "A new column can be explicitly added by setting the option `mergeSchema` to `true`."

Question 10: Which Data Manipulation Language (DML) commands are supported by Delta Lake to ensure ACID guarantees?

- A. Only `INSERT` (append) and `SELECT`.
- B. `INSERT`, `SELECT`, and `VACUUM`.
- C. `UPDATE`, `DELETE`, and `MERGE`.
- D. `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`.

Answer: C. `UPDATE`, `DELETE`, and `MERGE`.

Explanation: Besides basic appends and reads, "Delta Lake supports UPDATE, DELETE, and MERGE commands". These operations "ensure ACID guarantees." Examples of `UPDATE` and `DELETE` are provided.

Reference: Module 12 - Cloud Data Lakes - "Delta Lake supports UPDATE, DELETE, and MERGE commands" / "They ensure ACID guarantees."

Question 11: How can you query a previous version (e.g., version 2) of a Delta Lake table using Spark?

- A. `spark.read.format("delta").option("versionAsOf", "2").load(deltaPath)`
- B. `spark.sql("SELECT * FROM deltaTable@v2")`
- C. `spark.read.format("delta").load(deltaPath).history(2)`
- D. By restoring a backup corresponding to version 2.

Answer: A. `spark.read.format("delta").option("versionAsOf", "2").load(deltaPath)`

Explanation: Time travel allows querying previous versions. "You can query previous versioned snapshots of a table by using the DataFrameReader options `versionAsOf` and `timestampAsOf`." The example shows reading version 2 using `option("versionAsOf", "2")`.

Reference: Module 12 - Cloud Data Lakes - "You can query previous versioned snapshots of a table by using the DataFrameReader options `versionAsOf` and `timestampAsOf`." / (Code example showing `.option("versionAsOf", "2")`)

Question 12: Which data processing paradigm does Delta Lake unify, according to the slides?

- A. SQL Analytics and Business Intelligence.
- B. Data Science and Machine Learning.
- C. Streaming and Batch data processing.
- D. OLTP and OLAP.

Answer: C. Streaming and Batch data processing.

Explanation: One of the key features listed for Delta Lake is that it "Unifies streaming and batch data processing." This means the same table can be used as a sink for streaming jobs and a source for batch queries.

Reference: Module 12 - Cloud Data Lakes - (Delta Lake features list) / "Unifies streaming and batch data processing."

Question 13: What format are the commit files in the DeltaLog written in?

- A. Parquet
- B. Avro
- C. Plain Text
- D. JSON

Answer: D. JSON

Explanation: "Each commit is written out as a JSON file, starting with 000000.json." Subsequent commits generate files like 000001.json, etc.

Reference: Module 12 - Cloud Data Lakes - "Each commit is written out as a JSON file, starting with 000000.json."

Question 14: What is the primary purpose of defining a schema *before* data is stored in a Data Warehouse (schema-on-write)?

- A. To support unstructured data efficiently.
- B. To optimize storage costs for large datasets.
- C. To ensure data consistency and optimize for SQL analytics and BI performance.
- D. To enable direct reading by machine learning libraries.

Answer: C. To ensure data consistency and optimize for SQL analytics and BI performance.

Explanation: Data Warehouses are "Purpose-built for SQL analytics and BI." They use schema-on-write ("Data Warehouse defines the schema before data is stored") to structure data for efficient querying, using features like "schemas, indexes, caching, etc."

Reference: Module 12 - Cloud Data Lakes - "Purpose-built for SQL analytics and BI: schemas, indexes, caching, etc." / "Data Warehouse defines the schema before data is stored."

Question 15: According to survey results mentioned, what is a top challenge faced by data teams?

- A. Lack of powerful query engines.
- B. Data Quality and Staleness.
- C. Insufficient data volume.
- D. Difficulty migrating from Data Lakes to Data Warehouses.

Answer: B. Data Quality and Staleness.

Explanation: The slides highlight challenges: "60% reported data quality as top challenge." "86% of analysts had to use stale data...". "Getting high-quality, timely data is hard!"

Reference: Module 12 - Cloud Data Lakes - "60% reported data quality as top challenge." / "86% of analysts had to use stale data..." / "Getting high-quality, timely data is hard!"

Question 16: In a Lakehouse architecture, how can high SQL performance be achieved on data lake storage?

- A. By converting all data to CSV format.
- B. By relying solely on Spark's in-memory processing.
- C. Through optimizations like caching hot data, optimizing data layout within files, and using auxiliary structures like statistics and indexes.
- D. By moving all computation to the client-side.

Answer: C. Through optimizations like caching hot data, optimizing data layout within files, and using auxiliary structures like statistics and indexes.

Explanation: Achieving "Great SQL performance on Data Lake storage" involves several "Directly-accessible file storage optimizations": "Caching hot data in RAM/SSD", "Data layout within files to cluster co-accessed data", and "Auxiliary data structures like statistics and indexes".

Reference: Module 12 - Cloud Data Lakes - "Great SQL performance on Data Lake storage systems and file formats." / "Directly-accessible file storage optimizations can enable high SQL performance:" / "Caching hot data..." / "Data layout..." / "Auxiliary data structures..."

Question 17: What does an "add file" action within a DeltaLog commit record signify?

- A. A change to the table's schema.
- B. A new data file (e.g., a Parquet file) is now part of the current version of the table.
- C. A transaction ID for ensuring idempotency.
- D. Information about the commit for auditing purposes.

Answer: B. A new data file (e.g., a Parquet file) is now part of the current version of the table.

Explanation: A commit contains an array of actions. The "Add/remove file" action "adds/removes a file" from the table's tracked file list. For example, adding `1.parquet` and `2.parquet` would result in a commit with `add` actions for those files.

Reference: Module 12 - Cloud Data Lakes - "Each log record object... contains a commit, i.e., an array of actions..." / "Add/remove file: adds/removes a file" / (Example showing add actions for 1.parquet and 2.parquet)

Question 18: Which Spark DataFrame API enables optimizations by compiling operations into relational algebra?

- A. RDD API(`map` , `filter` , `reduceByKey`)
- B. Dataset API(`as[CaseClass]`)
- C. Spark DataFrame API itself (using declarative operations like `select` , `filter` , `groupBy`)
- D. Spark Streaming API(`DStream`)

Answer: C. Spark DataFrame API itself (using declarative operations like `select` , `filter` , `groupBy`)

Explanation: The slides mention that "New declarative interfaces for I/O enable further optimization." As an "Example: Spark DataFrame API compiles to relational algebra," allowing the Catalyst optimizer to improve execution plans.

Reference: Module 12 - Cloud Data Lakes - "New declarative interfaces for I/O enable further optimization." / "Example: Spark DataFrame API compiles to relational algebra."

Question 19: How can you read a Delta Lake table using Spark SQL after registering it as a temporary view named `loans_delta` ?

- A. `spark.read.format("delta").load("loans_delta")`
- B. `spark.sql("SELECT * FROM loans_delta")`
- C. `sc.textFile("loans_delta")`
- D. `spark.table("loans_delta").collect()`

Answer: B. `spark.sql("SELECT * FROM loans_delta")`

Explanation: The code example shows reading and querying the Delta table directly using Spark SQL after it has presumably been registered. `spark.sql("SELECT count(*) FROM loans_delta").show()` and `spark.sql("SELECT * FROM loans_delta LIMIT 3").show()` demonstrate this.

Reference: Module 12 - Cloud Data Lakes - (Code example `spark.sql("SELECT count(*) FROM loans_delta").show() )` / (Code example `spark.sql("SELECT * FROM loans_delta LIMIT 3").show() )`)

Question 20: What is the benefit of using the DeltaLog for data lineage?

- A. It automatically encrypts the data files.
- B. It provides a verifiable history of all changes, useful for governance, auditing, compliance, and debugging.
- C. It compresses the data files to save storage space.
- D. It ensures the data can only be read by authorized users.

Answer: B. It provides a verifiable history of all changes, useful for governance, auditing, compliance, and debugging.

Explanation: "The Delta Lake DeltaLog offers users a verifiable data lineage." This "is useful for governance, audit and compliance purposes" and "can also be used to trace the origin of an inadvertent change or a bug".

Reference: Module 12 - Cloud Data Lakes - "The Delta Lake DeltaLog offers users a verifiable data lineage." / "It is useful for governance, audit and compliance purposes." / "It can also be used to trace the origin of an inadvertent change or a bug..."